

UAM SAML Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAM SAML Implementation Guide.....	3
2. Introduction to UAM SAML Implementation.....	4
3. UAM SAML Implementation.....	9
3.1. UAM as SAML Identity Provider Components.....	10
3.2. UAM SAML Features.....	14
3.3. Implementation Planning.....	18
4. UAM as SAML Identity Provider.....	22
4.1. SAML Web SSO Configuration.....	23
4.2. Integration with SAML Service Providers.....	42
4.2.1. UAM SAML with Office 365.....	43
4.2.2. UAM SAML with Google.....	52
4.2.3. UAM SAML with FortiGate SSL VPN.....	55
4.2.4. UAM SAML with JIRA.....	68
4.3. User Entitlement and Identity Mapping.....	72
4.4. Custom Identity Provider Front End.....	82
5. UAM SAML Service Provider ACS Deployment.....	89
5.1. Deployment.....	91
5.2. Configuration.....	93
6. UAM SAML Reports.....	102
7. Appendices.....	103
8. Quick Setup Cheatsheet.....	108

1. Preface - UAM SAML Implementation Guide

This document provides the concept and implementation steps of AccessMatrix Universal Access Manager (UAM) server as a Security Assertion Markup Language (SAML) Identity Provider (IdP) and as a SAML Service Provider (SP). It also provides the configuration and deployment details of SAML SP Assertion Consumer Service (ACS) web application.

Intended Reader

It is intended as a reference for the security administrators and i-Sprint's Professional Service personnel to carry out the following operations:

- Configure the Identity Provider
- Configure the Service Provider
- Assign user entitlement
- Set up Identity Provider Front End

Document Scope

This document only provides the implementation steps to configure AccessMatrix UAM as a SAML IdP and AccessMatrix UAM as a SAML SP, as well as to configure UAM SAML SP ACS.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- [UAM Administration Guide](#)
 - [AccessMatrix Common Administration Guide](#)
 - [AccessMatrix Common Login Page Deployment Guide](#)
-

2. Introduction to UAM SAML Implementation

This document provides the information on the implementation of AccessMatrix UAM as SAML IdP.

Understanding SAML Web SSO

Security Assertion Markup Language (SAML) is an XML standard that allows secure web domains to exchange user authentication and authorization data. Using SAML, an online service provider can contact a separate online identity provider to authenticate users who are trying to access the secured content. With SAML, you can enable a single sign-on (SSO) experience where a user authenticates to one web application and without additional authentication, is able to access resources at another web application.

In a SAML model, there are mainly two parties involved:

- **Identity Provider (IdP)**
 - Manage and control usernames, passwords and other corresponding information used to identify, authenticate and authorize users for web applications hosted by a service provider.
 - AccessMatrix Universal Access Management (UAM) can act as an identity provider to cloud services and other web applications that support SAML services.
- **Service Provider (SP)**
 - Typically a web application, such as Google Apps, Microsoft Office 365, Webex.
 - A company that has many existing web application(s) can tap on the SAML framework of AccessMatrix UAM to act as a service provider (SP), providing SAML SSO service for its end users to access its web-based applications.

SAML has three main components:

- **Assertion**

A package of information containing the user authorization data. There are three different types of assertion statements: authentication, attribute, and authorization decision.

 - Authentication assertion validates the user's identity.
 - Attribute assertion contains specific information about the user.
 - Authorization decision assertion identifies what the user is authorised to do.
-

- **Protocol**

Defines how SAML asks for and receives assertions.

- **Binding**

Defines how SAML assertions and/or protocol messages are conveyed in, or over, other communication protocols.

SAML Web SSO Integration

Below is a checklist to go through before integrating SAML Web Browser Single Sign-On (SSO) with Cloud Services or web applications that support SAML:

1. Is it Service Provider-initiated or Identity Provider-initiated?

Refer to [Service Provider-initiated vs Identity Provider-initiated](#) section for details.

2. What is the SAML Binding used?

Refer to [SAML Binding](#) section for details.

3. What is the role of AccessMatrix UAM system?

Refer to [Role of AccessMatrix UAM system](#) section for details.

4. Is there a requirement for extra attributes as part of assertion?

5. Is AccessMatrix UAM system the Identity Provider for the Cloud Services?

Service Provider-initiated vs Identity Provider-initiated

SAML Web Browser SSO Profile can be SP-initiated or IdP-initiated. This refers to who initiates the SSO process.

Service Provider-Initiated

SP-initiated SSO means a SAML Request is used in the SSO process. Below is the flow:

- End-user visits service provider, and then the service provider sends a SAML request to the identity provider. After the user has been authenticated, the identity provider sends a SAML response to the service provider.
 - End-user visits portal after authenticated and chooses the service provider to access, then the portal redirects to the service provider URL to trigger SAML process, and then the service provider sends a SAML request to the identity provider. The identity provider sends a SAML response to the service provider.
-



Note: Usually all Cloud services are SP-initiated.

Identity Provider-Initiated

In IdP-initiated SSO, service provider does not send SAML Request to the identity provider. Below is the flow:

- End-user visits portal after authenticated and chooses the service provider to access, then the identity provider sends a SAML response to the service provider.

SAML Binding

SAML Binding is a mapping of a SAML message onto standard messaging formats and/or communications protocols. It is the format in which data is transferred between SPs and IdPs.

Types of SAML Bindings:

HTTP Redirect and HTTP POST Bindings

- SAML Single Sign-On without backend channel communication
- HTTP Redirect and HTTP POST SAML bindings are more commonly used for cloud services.

SAML Artifact Binding

- Backend channel is required
- SAML Artifact Bindings is uncommon, as it requires backend channel to be established between the Service Provider and the Identity Provider.

Role of AccessMatrix UAM System

AccessMatrix UAM has the ability to act as both the Identity Provider or the Service Provider. As mentioned in earlier section, AccessMatrix has a number of components that helps to facilitate the SAML integration. In order to proceed with the integration, it is important to understand what the role of AccessMatrix UAM in this integration.

Is AccessMatrix UAM used as the Identity Provider in SAML?

This means the users reside in AccessMatrix UAM (either in default user store or in an external user store) and the user authentication is done by AccessMatrix UAM. **As an Identity Provider**, AccessMatrix

UAM will help to **read/understand the SAML request** and **generate the SAML response containing the user identity** to be given to the Service Provider. If this is what you aim to achieve, then proceed to [UAM as IdP](#) section of this document.

Is AccessMatrix UAM used as the Service Provider in SAML?

This means the users may/may not reside in AccessMatrix UAM and the user authentication is done by another system. **As a Service Provider**, AccessMatrix UAM will help to **generate the SAML request and parse/understand the SAML response containing the user identity**. This means AccessMatrix UAM will simply **trust** (after SAML validation) the identity of the user as given by another system, an Identity Provider. After acquiring the identity, depending on the type of integration with AccessMatrix UAM, AccessMatrix may either:

- Return the user identity to the API caller.
- Map the identity to find the user record in AccessMatrix UAM and issue an AccessMatrix session for the user.

If AccessMatrix UAM is to be used as both the Identity Provider and Service Provider, then you need to go through the next part of this document to set up AccessMatrix UAM as the Identity Provider and AccessMatrix UAM as the Service Provider.

Product Features of SAML in AccessMatrix UAM

Below is the list of AccessMatrix UAM product features and the supported SAML standards:

- **AccessMatrix UAM system as an Identity Provider's backend**
 - Service/Identity Provider initiated Web Browser SSO Profile (Redirect/POST Bindings)
 - Attribute Statement as part of Assertion
 - **AccessMatrix UAM system as a Service Provider's backend**
 - Identity Provider initiated Web Browser SSO Profile (Redirect/POST Bindings)
 - Identity Provider initiated Web Browser SSO Profile (SAMLArtifact)
 - Attribute Statement as part of Assertion
 - **Web Security Agent (WSA) as Service Provider SAML Assertion Consumer**
 - Identity Provider initiated Web Browser SSO Profile (POST Binding)
 - Attribute Statement as part of Assertion
-

- **SAML Service Provider ACS Servlet**

- Service Provider initiated Web Browser SSO Profile (Redirect/POST Bindings)
- Attribute Statement as part of Assertion

Need to get up and running fast?

To help you get started quickly, we've included a [Quick Setup Cheatsheet](#) that summarises the essential setup steps and configurations of typical usage cases. It's especially useful for fast reference or for teams working under tight timelines, or just want a high-level overview without diving into the full guide.

3. UAM SAML Implementation

The following pages provide detailed information on the components involved in the deployment of AccessMatrix UAM as an Security Assertion Markup Language (SAML) Identity Provider and Service Provider:

- [UAM as SAML Identity Provider Components](#)
- [UAM as SAML Service Provider Components](#)

3.1. UAM as SAML Identity Provider Components

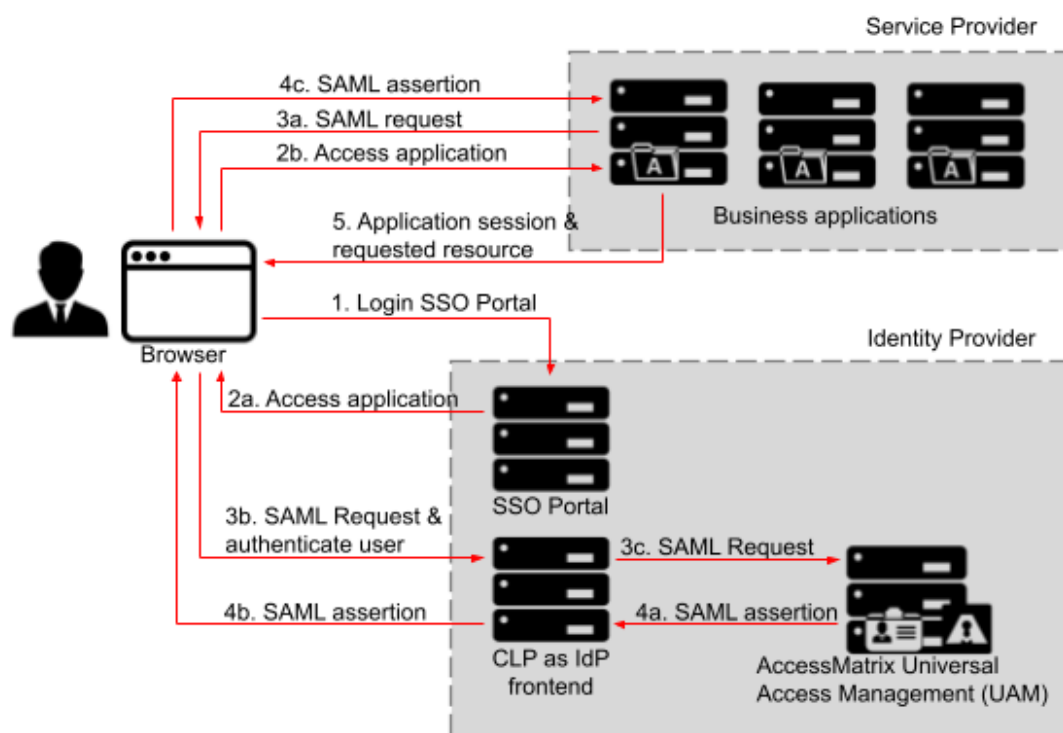
As SAML IdP, the AccessMatrix UAM system provides the identities, including the means to authenticate the users. It contains the user profile such as username, email, mobile, and so on. It also initiates the SAML response that is redirected to the Service Provider to assert the user's identity.

AccessMatrix SAML IdP Components

Below is the list of components involved when the AccessMatrix UAM acts as the identity provider:

- SSO Portal to single sign-on to user's entitled SAML applications
- Common Login Page (CLP) as the identity provider's front end
- AccessMatrix UAM system as the identity provider's back end
- Business applications as SP's SAML assertion consumer

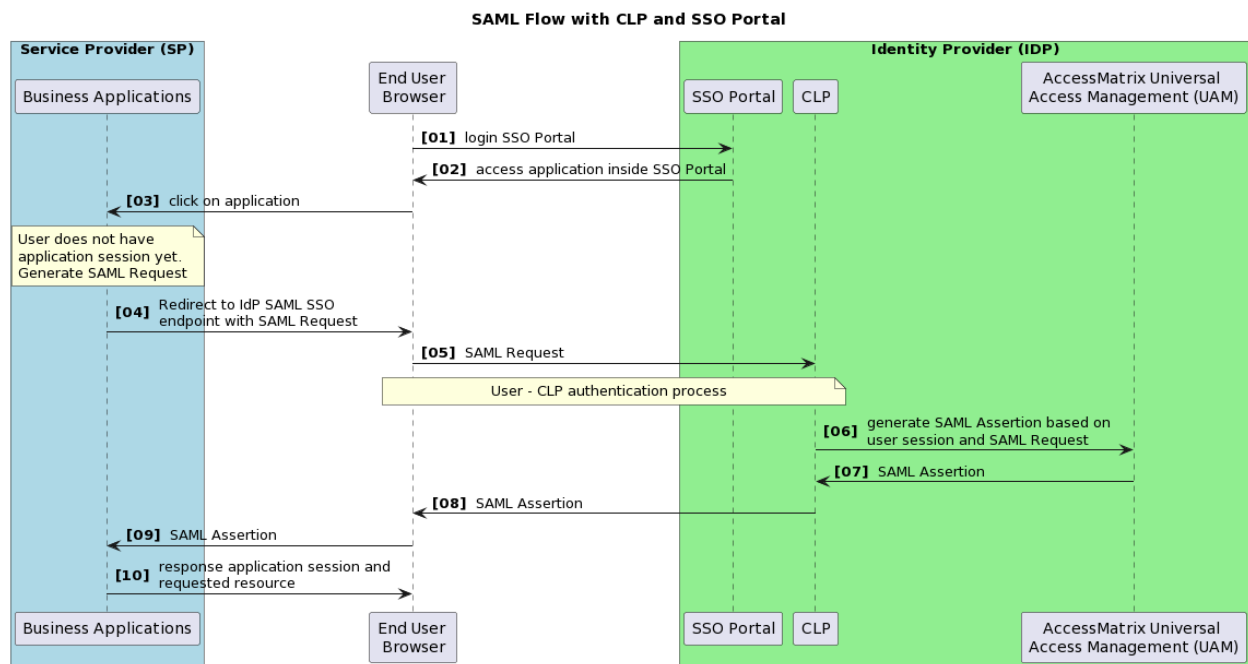
SAML Component Diagram with CLP and SSO Portal



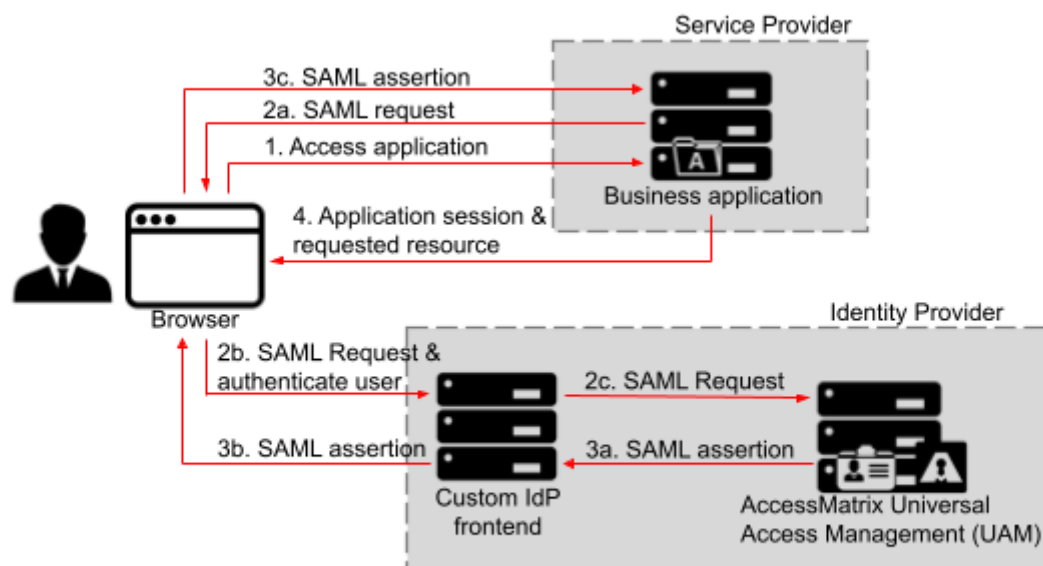
In the figure above:

1. The user logs in to SSO Portal.
- 2a. The SSO Portal displays the user entitled business applications.
- 2b. The user access the entitled business applications in SSO Portal.
- 3a. If the user is not authenticated yet, then the application directs the SAML request to the CLP (IdP front end) to log in.
- 3b. The user provides the credential and CLP authenticates the user.
- 3c. Once login is successful, CLP will direct the SAML request to the AccessMatrix UAM to process.
4. AccessMatrix UAM returns the SAML assertion back to the application through the CLP.
5. The application validates the assertion, then returns the application session and requested resources to the user.

SAML Sequence Diagram with CLP and SSO Portal



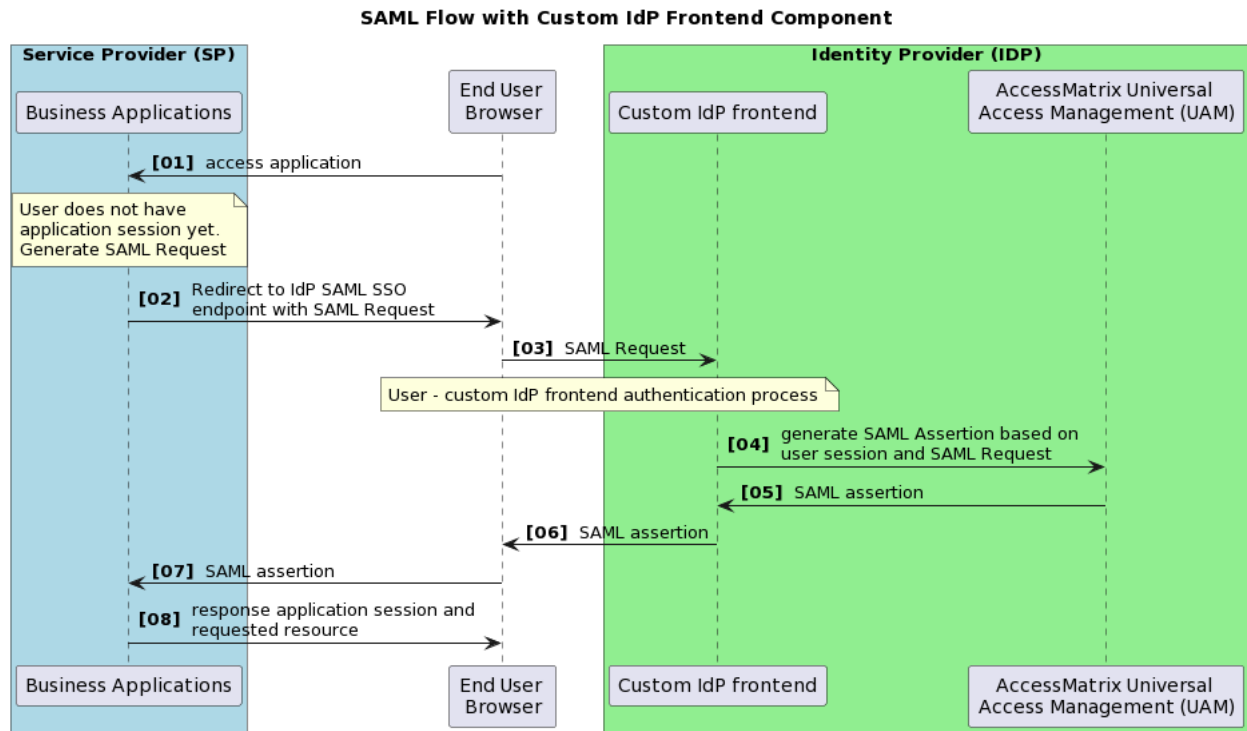
SAML Component Diagram with Custom IdP Front End Component



In the figure above:

1. The user access the protected business application through the browser.
 - 2a. If the user is not authenticated yet, then the application directs SAML request to the custom IdP front end to log in.
 - 2b. The user provides the credential and the custom IdP front end authenticates the user.
 - 2c. Once login is successful, the custom IdP front end will direct the SAML request to the AccessMatrix UAM to process.
 3. AccessMatrix UAM returns the SAML assertion back to the application through the custom IdP front end.
 4. The application validates the assertion, then returns the application session and requested resources to the user.
-

SAML Sequence Diagram with Custom IdP Front End Component



3.2. UAM SAML Features

The AccessMatrix UAM SAML solution supports the following SAML features:

Authentication Context Processing

AccessMatrix UAM supports the use of authentication contexts for SAML web SSO. Authentication contexts provide information about how a user is authenticated (i.e. the authentication methods used) at the Identity Provider (IdP). By specifying the expected authentication context(s), you can require users to be authenticated using those authentication method(s). This is useful in situations where it is necessary to conform to specific security requirements.

In general, authentication context processing works as follows:

1. When the Service Provider (SP) creates an authentication request, it includes authentication contexts in the request and sends it to the IdP.
2. The IdP prompts the user for authentication using authentication methods based on the contexts requested.
3. After the user is authenticated at the IdP, the IdP compares the authentication contexts for each of the authentication methods used against the requested contexts.
4. If the contexts used match the requested contexts, they are added to the assertion response that is sent to the SP.
5. The SP then compares the contexts received with the requested contexts.
6. If they match, the authentication is successful.

Performing SAML web SSO with authentication context processing varies depending on the role UAM is playing in the SAML SSO flow and who initiates the SSO:

UAM as Identity Provider

When UAM acts as the IdP, both SP-initiated and IdP-initiated web SSO are available. Each starts off differently:

- **SP-initiated SSO**

In this scenario, the SP generates a request that includes the authentication context(s) and sends it to the AccessMatrix (acting as the IdP).

- **IdP-initiated SSO**

In this scenario, instead of the SP generating a request, AccessMatrix (acting as the IdP) initiates the SSO flow. To obtain the required authentication contexts, it refers to the contexts specified within the **Response Authentication Context Mapping** attribute configuration in the [SP created in AccessMatrix \(using the "UAM as Identity Provider" implementation\)](#).

Next, the user is authenticated using the login module(s) specified in the authentication realm configured in AccessMatrix. The context(s) associated with the login module(s) are matched with the context(s) in the request, and if successful, they are included in the response to the SP. The SP then checks the response and compares the contexts included with those requested.



Note: Refer to the UAM as IdP [Enable Authentication Context Processing](#) section for the instructions on how to enable authentication contexts.

UAM as Service Provider

When UAM acts as the SP, only the SP-initiated SSO flow is available. In this scenario, UAM (as the SP) generates a request that includes the authentication context(s) and sends it to the IdP. The context(s) included in the request are retrieved from different configurations depending on your deployment:

- **When using the UAM SAML Service Provider Assertion Consumption Service (ACS):**

The authentication context(s) to be included in the request are based on the **Response Authentication Context Mapping** attribute configuration in the [SAML web browser SSO lookup and login module](#) in AccessMatrix.

- **When using REST API:**

The authentication context(s) to be included in the request are based on the **Response Authentication Context Mapping** attribute configuration in the [SP created in AccessMatrix \(using the "UAM as Service Provider" implementation\)](#).

Next, the request is sent to the IdP, where the user is authenticated. The context(s) associated with the authentication methods used are matched with the context(s) in the request, and if successful, they are included in the response to the SP. The SP then checks the response and compares the contexts included with those requested.



Note: Refer to the UAM as SP [Enable Authentication Context Processing](#) section for the instructions on how to enable authentication contexts.

SAML ForceAuthn for Session Integrity & Step-Up Authentication

What is Session Integrity & Step-Up Authentication?

Session Integrity means making sure the person using the system now is still the right person and not someone else using an open session, eg, the user walks away from their computer.

Step-Up Authentication seeks an extra layer of security by requiring a stronger or fresh login before granting access to more sensitive actions.

eg, A simple login(like entering a password) is enough for a user to sign in to an app. But if the user tries to transfer money, the system may ask for proof of identity again, even though the user is signed in, with something stronger(eg, token otp).

SAML ForceAuthn

In normal single sign-on (SSO), once a user signs in, the system remembers the user for a while. That means when a user visits another connected app, no user login is needed.

However, there are times when the system should not rely only on that remembered session.

This is where ForceAuthn comes in. It is a special flag in SAML that tells the identity provider (the login system) to ignore any existing session and ask the user to log in again, even if they already have a valid session.

The benefits of SAML ForceAuthn include having an extra layer of security, helping prevent misuse of stale or hijacked sessions, supporting step-up authentication for high-risk actions, and building trust with users by protecting their data.

UAM as Identity Provider

UAM recognises and honours the ForceAuthn feature. It will prompt the user to re-enter their credentials, even if they're already signed in. This helps maintain session integrity and supports step-up authentication.

This can be used in conjunction with the Identity Provider Policy by configuring in [Policy Tab](#) for more advanced use cases. Such as enforcing that users have to use a specific method (such as OTP) for step-up authentication when prompted to re-enter their credentials.

Available in both SP-initiated and IdP-initiated web SSO. Each is slightly different.

- **SP-initiated SSO**

In this scenario, the SP generates a request that contains ForceAuthn=true and sends it to AM (acting as the IdP). AM will honour this request from SP, and prompt the user to re-enter their credentials.

- **IdP-initiated SSO**

In this scenario, instead of the SP generating a request, AM (acting as the IdP) initiates the SSO flow, hence there is no authentication(SAML) request to contain ForceAuthn.

The feature is activated by setting **Always Prompt Re-authentication for IdP-initiated SSO** to true in the SP configuration.

The session from re-authentication is valid for IdP-initiated SSO for 30 seconds; after the time lapses, the session cannot be used for SSO, and the user will need to re-authenticate to get a new session for SSO

UAM as Service Provider

UAM supports the ForceAuthn feature. It can generate authentication requests containing forceAuthn. This will trigger the IdP (if it recognises and honours ForceAuthn) to ask the user to log in again with ForceAuthn=true.

The feature can be activated by

- Setting **Default 'forceAuthn' in SAML Request** to true in the SP(UAM as SP) configuration
- calling REST API authz/saml/authnReq/generate request with "params":{"forceAuthn": true}
- API request parameter has higher precedence and will override the value of 'forceAuthn' if there is a conflict between the 2 methods

3.3. Implementation Planning

Implementation Checklist for UAM as SAML IdP

Below is the checklist for implementing AccessMatrix UAM as SAML IdP.

Tasks	Reference
Configure SAML IdP	SAML Identity Provider (IdP)
Configure SAML SP	SAML Service Provider (SP)
Edit SAML Configuration	SAML Configuration
Configure user entitlement and identity mapping	User Entitlement and Identity Mapping
Set up IdP Front End	IdP Front End

Implementation Planning

During implementation, you need to understand the customer's nature of a business, studying the customer's requirements and its existing environments to carry out the action plans. The following sections discuss the implementation considerations that should be assessed during planning.

Security Consideration

- **Certificate and Key Pair**

The most important thing to secure is the Identity Provider's certificate and private key because in SAML, the trust relationship between the Service Provider and Identity Provider is based on the signature in SAML Response. Thus, ensure that the keypair are generated securely, especially when generating for the production environment. Testing or staging environment must have a different key pair from the production environment as part of security best practice. In SAML, the certificate does not have to be signed/issued by a Certificate Authority (CA). Normally, the certificate exchange or configuration itself specifies the exact certificate to be used in the trust relation between the Identity Provider and Service Provider. Thus, it is not based on trusted CA. Keystore file containing private key must be secured from the OS permission perspective, for example, to allow only the OS user that is used to run AccessMatrix Server to have read access of the keystore. Keystore file that contains private key, should not be given to a Service Provider. If you need to provide the Identity Provider certificate, ensure that you provide the certificate file,

not the keystore file. Keystore file is also normally secured by a keystore password. When you configure in AccessMatrix UAM, specify a keystore and its password. The password will be encrypted when it is stored in the database.

- **Signature Algorithm**

The SAML Response, generated by AccessMatrix UAM as Identity Provider, contains a signature to sign the whole content of SAML Response/Assertion to ensure that the content is not tampered. It is best to use RSA_SHA256 as the signature algorithm, which is the default value in AccessMatrix UAM, and avoid using RSA_SHA1 as SHA-1 is no longer considered as secure for signing purpose.

- **SAML Response Encryption**

SAML Response can optionally be encrypted, especially if the SAML Response contains a sensitive information that should be known only between the Identity Provider and Service Provider. It is not common to encrypt the SAML Response since normally it contains only the user identity for the SSO to happen. Many cloud services do not support encryption for SAML Response. If SAML Response encryption is required, it is best to run AccessMatrix UAM on Tomcat instead of Weblogic and Websphere to reduce the likelihood of library conflicts.

- **Expiry of SAML Response**

Each SAML Response has a validity period. The length of the validity is configurable in AccessMatrix UAM. Normally, SAML SSO process happens very fast especially the time between SAML Response generated by the AccessMatrix UAM and then verified by the Service Provider. In most cases, it happens well within one-minute since there is no user interaction. However, we also need to consider the possibility of out-of-sync clock. Thus, setting it to 1-2 minutes is good enough. AccessMatrix UAM has a default value of 1 minute for SAML Response validity.

- **Cross-Site Scripting (XSS) for Custom Front End**

If you need to implement a custom IdP front end, the front end will need to handle the SAMLRequest and RelayState parameters provided by the Service Provider. As part of good security practices, if you need to render the values in the front end, the values must be properly encoded/escaped as needed.

Performance Consideration

In terms of performance, the majority of the processing for SAML Identity Provider is when it needs to sign the SAML Response. This step uses computing power considerably due to the use of asymmetric private key.

- **Key Size**

Although longer key length increases its security strength, it also comes with a performance cost. Currently, the norm is to use RSA with 2048-bit key length. Using a higher bit, such as 4096-bit will slow down the performance significantly.

- **Encryption**

Encryption increases the processing time significantly, although not as much as the signing process. Thus, ensure that the server has enough capacity based on the expected load.

AccessMatrix UAM as the Identity Provider means that it needs to authenticate the user, which in most deployments, involves two-factor authentication. In addition to that, the SAML SSO uses a significant amount of computing power.

Integration Consideration (Identity Provider Front End)

Integration with AccessMatrix UAM as SAML Identity Provider has a few options.

- **AccessMatrix UAM server**

AccessMatrix UAM server is the Identity Provider back end that includes user authentication and also the SAML processing (parsing SAML Request and generating SAML Response). This component is required across options. The difference of the options are on whether Common Login Page (CLP) web application is used as the IdP front end or whether a custom IdP front end is used instead.

- **Common Login Page (CLP) as IdP front end**

AccessMatrix UAM includes the CLP web application that can optionally be used. CLP's main function is to do user authentication and SSO to the target application. CLP supports some authentication methods and it acts as the login page for other AccessMatrix web applications, such as SSO Portal, UAS Token Activation Portal (UAS TAP), etc. For federated SSO, CLP supports SAML and OpenID Connect (OIDC) standards, wherein CLP acts as the Identity Provider/OpenId Provider front end. In the context of SAML Identity Provider, with the use of CLP, normally, there is no need to call APIs to do the SAML SSO process.

However, some deployments may have more specific requirements in their SSO and authentication processes, or they may have certain preferences regarding the user interface/experience that cannot be achieved by CLP. In order to satisfy these requirements, a custom IdP front end is required. In such cases, the custom front end can integrate with the same AccessMatrix UAM server as the back end. Such integration is supported both for SAML and OIDC, if required.

- **Custom IdP Front End**

Custom IdP front end can easily integrate with AccessMatrix UAM via its web service API for user authentication and the generation of the SAML Response. A custom IdP front end has flexibility in terms of having its own flow or process for user authentication and SSO. It can make use of AccessMatrix's SAML capabilities to seamlessly SSO between applications with a mix of normal Web SSO (same domain) and Federated SSO (SAML and OIDC). For more details on the flow and required APIs, refer to [Custom IdP Front End](#).

4. UAM as SAML Identity Provider

The following pages provide detailed instructions on the configurations needed for AccessMatrix UAM to function as the SAML Identity Provider:

- [SAML Web SSO Configuration](#)
- [User Entitlement and Identity Mapping](#)
- [Custom Identity Provider Front End](#)
- [Integration with SAML Service Providers](#)

4.1. SAML Web SSO Configuration

Setup Steps

Prior to SAML Identity Provider (IdP) configuration, it is assumed that the AccessMatrix UAM Server has been properly set up. Otherwise, the administrator should first proceed to set up the AccessMatrix UAM Server. Refer to the [AccessMatrix Common Deployment Guide](#).

Having ensured that the UAM Server has been properly installed, the administrator may proceed to perform the following configuration steps:

- [Upload SAML License](#)
- [Set up Identity Provider Front End](#)
- [Configure SAML Identity Provider \(IdP\)](#)
- [Configure SAML Service Provider \(SP\)](#)
- [Edit SAML Configuration](#)

Upload SAML License

Before doing any configuration in the Admin Console, secure the server license required to implement AccessMatrix UAM as SAML IdP. Ensure that **UAM FEATURE** is enabled and a value for the **SAML: Service Provider** license feature is specified.

To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
 2. Select the **Server Id** and click **Edit**.
 3. Select **Licensed Features** tab and select the first radio button. Browse for the license file.
 4. Click **Upload Licence** File to upload the license file to the server.
 5. Save the changes.
 6. Restart the AccessMatrix server for the new license to take effect.
-

Set Up Identity Provider Front End

An Identity Provider front end defines the way a user is identified and validated using user credentials as part of a login flow. In AccessMatrix UAM as IdP implementation, there are 2 methods that can be done:

- Using Common Login Page (CLP), or
 - First, deploy CLP as described in [Deployment Alternatives](#).
 - Then, set up CLP as SAML IdP as described in [Set Up CLP as SAML Identity Provider](#).
- Using Custom Identity Provider Front End

Refer to the [Custom IdP Front End](#) section for the details.

Configure SAML Identity Provider (IdP)

Note: ' * ' denotes a mandatory attribute.

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Identity Provider** and click the **Create** button.
2. Select the "UAM as Identity Provider" implementation and click **Next**. The **Create Identity Provider** page is displayed.
3. Fill in the basic IdP information:

Attribute Name	Description
*Identity Provider Id	Unique Id of Identity Provider, e.g. "MyIdP".
*Identity Provider Name	Name of Identity Provider, e.g. "Identity Provider".
Description	Description of Identity Provider.
*Id used as issuer	A kind of Id, normally in the URL form to represent the Identity Provider entity.

4. You may fill in the values of the following attributes. These attributes are applicable when the AccessMatrix UAM acts as a SAML IdP:

General Tab
Authentication Request

General Tab	
Allowed Signature Algorithm	Allowed signature hash algorithms for validating signed request/artifact-resolve. Note: This attribute is only used by the AccessMatrix server that acts as an Identity Provider. Options: ○ "SHA1" ○ "SHA256" ○ "SHA384" ○ "SHA512" Default: SHA256, SHA384, SHA512
Authentication Response	
Assertion validity time (in seconds)	The validity of SAML Response and SAML Response's Assertion is based on a time-based condition which specifies the validity time range. When AccessMatrix UAM system is IdP, upon generating SAML Response, this attribute value would be used to derive 'notOnOrAfter' time condition of the Assertion Subject and the Response Subject. Time is always in UTC timezone. The time unit of this attribute is in seconds. Default: 60
Assertion Start Time Validity Adjustment (for IDP initiated response only)	For Identity Provider-initiated Response, when SAML Response is generated, its Assertion Subject's 'NotBefore' time is generated based on the issue time of the Response. Sometimes the SP server time and Identity Provider server time may not be in sync and this will fail SP to use the SAML Response. To resolve this issue, this configuration is introduced to allow buffer time to be included (Assertion Subject's 'NotBefore' time = 'issueTime' - Assertion Start Time Validity Adjustment (for Identity Provider-initiated response only)) to Assertion Subject's 'NotBefore' time. Time is always in UTC timezone. The time unit of this attribute is in seconds. Default: 60
Web Browser SSO Tab	
Service URLs	
*Single Sign-On Service URLs	The Identity Provider service URL to receive the Authentication request or Single Sign-On request from the Service Provider. This value is used to specify the destination of the SAML Request. URL format: <HTTP Binding> - <SSO URL> ○ The URL with no binding is taken as a wildcard during API call, i.e. it can be taken as both HTTP-POST/Redirect. This URL with no binding will be exported as HTTP-POST binding during metadata export.

General Tab	
	For the new entry URL that is both POST & Redirect, add two entries of different bindings using the same URL. Example: <POST> - <URL1> <Redirect> - <URL1>
Authentication Request	
Acceptance time (in seconds)	This setting is to control the validity of an authentication request; the difference between the time the authentication request is issued and the time the authentication request is received must be within this configured time. Default: 360
Authentication Request Required Signing	Enable this setting if SAML Request requires signing. Default: false
Artifact Resolution Tab	
Artifact Resolve Service URLs	The Identity Provider service URL(s) to receive the Artifact Resolve request from the Service Provider.
Digital Signature Tab	
Keystore	The path to the signing keystore file in JKS format. The key pair(s) in the keystore is/are used for signing the SAML Response generated by the Identity Provider.
Keystore Passphrase	The password of the keystore.
Active Key Alias	The active key for signing.
Active Key Certificate's Expiry Date	The expiry date of the active key certificate.
Signature Algorithm	The signing algorithm used for SAML signature. Default: RSA_SHA256.
Log Expired Certificate	If enabled, an error log is added when an expired certificate from the keystore is used. This has no impact on the usage of the certificate, as the operation is still allowed to proceed even if the certificate has expired. Default: false
Key Rollover	

General Tab	
New Key Aliases	List of key aliases to use for key rollover. Aliases are used in order from the top of the list to the bottom for key rollovers. Usage of aliases restarts from the top of the list once again after reaching the bottom of the list. If there is currently no existing value configured for the Active Key Alias attribute, the first alias in the list is used.
Coexisting Key Alias	The new key to be rolled over to in the key rollover event. The value of Active Key Alias will be updated with this after the rollover event.
Enable Coexisting Key	When enabled, the coexisting public key string will be included in the exported metadata. The coexisting public key string will be identified as the "New Key" and the active public key string will be identified as the "Active Key". Default: false
Date to Perform Key Rollover	Date to start the signing key rollover process. Upon reaching the specified date, the value of Coexisting Key Alias is updated to the next unused alias in the New Key Aliases list. The use of the coexisting key is enabled. Both the coexisting key and active key public key string are included in the exported metadata. Note: The Start Key Rollover button must be clicked for the above to take effect.
Start Date of Coexisting Key Usage	Date to complete the signing key rollover process. Upon reaching the specified date, the value of Active Key Alias is updated to that of Coexisting Key Alias . Enable Coexisting Key is set to "false" and only the updated active key is used for signing. Only the active key public key string will be in the exported metadata. Note: The Complete Key Rollover button must be clicked for the above to take effect.

Prepare Information Required by the SP

This section contains the information that has been configured in the IdP and needs to be given to the SP.

Common Configuration

- **SAML IdP issuer ("Id used as issuer" attribute)**

To denote the identity of the Identity Provider.

- **SAML IdP SSO URL ("Single Sign-On Service URLs" attribute)**

Sometimes it is called login URL or SSO login URL. This URL is given by the Identity Provider used

to authenticate the user. Identity provider will look for valid login credentials or check for valid session cookies.

- **IdP Certificate ("Keystore" attribute)**

It is used to trust SAML messages originating from Identity Provider software.



Note: The administrator is required to extract the certificate from the keystore and only provide it, not the whole keystore to the SP.

Advanced Configuration

- **Authentication Request Required Signing**

Requires to be enabled if the SAML Request from service provider requires signing.

- **Signature Algorithm**

Change the signature algorithm for SAML signing. Default is RSA_SHA256.

The above information can also be exported using **Export Metadata** as described below.

Export IdP Metadata

Once SAML IdP is created, the **Export Metadata** button will be displayed. This button exports the metadata into an XML file for loading to the Service Provider. Below is the filename format:

<Id used as issuer>-metadata.xml

This SAML metadata will be used to share configuration information between the IdP and the SP.

Roll Over Key Pair for Signature (IdP)

AccessMatrix UAM supports the rollover of SAML signing keys with zero downtime. Zero downtime is achieved through the use of a coexisting key, which is the new key that will be rolled over to at the completion stage.

The signing key rollover process involves two stages:

1. **Start Stage – Preparation for Service Provider (SP) client application**

The start stage is triggered by clicking the **Start Key Rollover** button after the specified **Date to Perform Key Rollover** is reached. Doing so allows the new key to be included as the additional key (the coexisting key) during the Identity Provider (IdP) metadata export. This is to prepare the SP of the corresponding application for the use of this new key for signature verification.

2. **Completion stage – Activation of new key as active key**

The completion stage is triggered by clicking the **Complete Key Rollover** button after the specified **Start Date of Coexisting Key Usage** is reached. This causes the new key to be used as the active signing key.

Follow the steps below to perform the key rollover:

1. Under the **Keystore** attribute, configure the path to the signing keystore file containing the current active key and new keys for key rollover.
2. Under **New Key Aliases**, enter the aliases of the keys in the keystore. The current active key must be at the top of the list, followed by the next key(s) for rollover.
3. Configure **Date to Perform Key Rollover** to specify the date of the start stage of the key rollover.
4. Click the **Start Key Rollover** button after the **Date to Perform Key Rollover** is reached to start the key rollover.
5. Configure **Start Date of Coexisting Key Usage** to specify the date of the completion stage.
6. Click the **Complete Key Rollover** button after the **Start Date of Coexisting Key Usage** is reached to complete the key rollover process. The new key is now used as the new active key.

Configure SAML Service Provider (SP)

The following sections provide the steps and the list of attributes relevant for SP configuration when AccessMatrix UAM is the IdP. There are two ways to populate the SP configuration:

- Using **Import Metadata** button;
- **Manual configuration** by filling in the relevant attributes

However, prior to SP configuration, there is a need to get the required information from the service provider as discussed below.

Obtain Information Required by the SAML SP

This section contains the information that is required to be configured in the Admin Console's Service Provider:

Common Configuration

- **SAML SP issuer ("Id used as issuer" attribute)**

Service Provider provides a standard issuer URL or sometimes it allows the user to choose or use the domain-specific issuer.

- **Subject or Name Id Value**

The name Id value used in the SAML Response, where the user identity used to identify the user must be provisioned and referenced by both parties.

- **Assertion Consumer Service (ACS) URLs**

The Service Provider's URL to send the SAML Response to. Normally, the Service Provider will provide this information in the documentation or as part of its configuration. Some cloud services may provide a unique ACS URL for each account/tenant.

! **Important Note:** All of the information above should be provided by the Service Provider in its documentation. If they do not provide the SAML SP issuer or the ACS URL, one way to find out is to trigger the Service Provider to generate a SAML Request. Then, you may use the encoded SAML Request (normally found in the request parameter `SAMLRequest`). You may use a SAML decoding tool online to decode it into the actual XML payload. The Issuer and ACS URL are normally specified in the XML payload.

Advanced Configuration

- **Whether SP signs the request**

If it does, SP normally provides its certificate as part of the documentation, metadata, or configuration. You should store the certificate in a keystore and configure it under the **Digital Signature** tab (the **Keystore** attribute and its related attributes).

- **Whether SP wants the assertion be encrypted**

If SP requires that the assertion be encrypted, it will provide the certificate for encryption as part of the documentation, metadata, or configuration. You should store the certificate in a keystore and configure it under the **XML Encryption** tab (the **Keystore** attribute and its related attributes). You also need to set **Assertion Encryption** to true.

- **Response signature scope**

Most SP requires that IdP sign the whole SAML Response. However, some SP may strictly require only the Assertion part to be signed. In this later case, the **Response signature scope** attribute should be set to "Assertion". If no information is provided by the SP, it is generally safe to leave the value to default ("Response").

Default is Response.

- **Extra user information for Assertion**

Some SPs may require additional attributes to be provided (e.g. to provide the user name, user group, department, email). In this case, Accessmatrix UAM as IdP can provide the extra attributes as part of the SAML Response assertion through the following ways:

- AccessMatrix UAM system's User Response configuration under Admin Console's SP configuration
- Passing of attribute key pair to REST API as a parameter (only for IdP-initiated web browser SSO Profile)



Note: It is not commonly used for cloud services.

Import SP Metadata

During SAML SP configuration, the **Import Metadata** button is displayed. This button imports the metadata files for the Service Provider.

To import:

1. Click the **Import Metadata** button. The **Import Metadata for Service Provider** wizard is displayed.
2. Click the **Browse...** button to locate the metadata file. The filename format is **<Id used as issuer>-metadata.xml**.
3. Click the **Upload Metadata** button to upload the file.
4. Then, click the **Save** button.

Configure Service Provider (SP) in Admin Console



Note: ' * ' denotes a mandatory attribute.

1. To configure the SAML SP, navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Service Provider** and click the **Create** button.
 2. Select the "UAM as Identity Provider" implementation and click **Next**. The **Create Service Provider** page is displayed.
 3. Fill in the basic SP information:
-

Attribute Name	Description
*Service Provider Id	Unique Id of Service Provider, e.g. "MySP".
*Service Provider Name	Name of Service Provider, e.g. "Service Provider".
Description	Description of Service Provider.
*Id used as issuer	A kind of Id, normally in the URL form to represent the Service Provider entity.

4. You may fill in the values of the following attributes. These attributes are applicable when the AccessMatrix UAM acts as a SAML IdP:

General Tab	
Assertion	
*Assertion Consumer Service URLs	SP's URL to consume SAML assertion. This configuration is needed when the AccessMatrix UAM system is an identity provider or a service provider, to specify the recipient of the SAML Response.
Name Id	
Note: Either Value or both Value and User Response to return as Nameid value attributes have to be mandatory.	
Format	A constant to describe the Name ID value and it is used to export SP metadata. The important thing is to agree on the Name ID format between IdP and SP. Possible values: - o "Email" (urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress) - o "Persistent" (urn:oasis:names:tc:SAML:2.0:nameid-format:persistent) - o "Unspecified" (urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified) - o "Transient" (urn:oasis:names:tc:SAML:2.0:nameid-format:transient) Default: Unspecified
Value	The value to be retrieved from the user object as user identity. If a user's identity is to be retrieved from User Response (e.g. user's UUID), configure this attribute to "User Response" instead. Possible values: o "Id"

General Tab	
	○ "Name" ○ "DN" ○ "Email" ○ "Secondary Account" ○ "User Response"
User Response to return as NameID value	The value to be retrieved from the User Response as user identity. This setting is used when Name ID Value is set as "User Response". User Response configuration can be chosen as flexibility in specifying what value to return as Name ID value. Specify only one field/attribute in the User Response configuration. If it contains more than one value, only one (randomly chosen) will be used.
Attributes	
Attributes to return in the assertion	Upon successful authentication, the specified set of user information to be sent to this service provider as part of the assertion. User Response configuration is used to specify this information. Note: SAML Assertion allows add-on payload to include user attributes or other information from the identity provider to service provider.
Response	
Response signature scope	It is used to define the scope of the SAML Response to be signed, e.g. Response, Assertion, etc. Possible values: ○ "Response" ○ "Assertion" Default: Response
Assertion Encryption	Indicate whether to apply encryption to SAML Response's assertion. Note: This attribute is only used by the AccessMatrix server that acts as an Identity Provider. Default: false
Access Control	
Allow All Users to Access	Enabling this feature will allow all authenticated users to access this Service Provider. To control user access to the web applications or web services of a Service Provider, this setting must be disabled. To control user access, go to the SAML Application Privilege tab of the View User page and define which service provider is accessible for that particular user. Default: true
Others	

General Tab	
Response Authentication Context Mapping	This feature is used to enable the use of authentication context processing. Specify an authentication context here to require the user to be authenticated using that authentication context for IdP-initiated SAML use cases. Multiple contexts can be specified. The specified context(s) will serve as the requested authentication context(s) and be used in the validation of the SAML response; authentication will fail if the expected authentication context(s) is/are not found.
Allow Multiple Profiles	This is to allow a user to have multiple login profiles (a secondary account) for the same application. Note: This attribute is only used by the AccessMatrix server that acts as an Identity Provider. Default: false
Is WSA Being Used?	A flag to indicate if WSA is being used. Default: false
Web Browser SSO Tab	
URL to initiate SSO	A URL that initiates SAML SSO operation. It is a URL to the servlet(s) for the SSO service or the SAML Authentication Request Service. Note: It is required only by Identity Provider portal or USO Agent.
Is IDP Initiated Flow	Indicate whether this SP is for IdP-initiated SSO.
Always Prompt Re-authentication for IdP-initiated SSO	Enabling this will require the user to re-authenticate, even if there is a valid session, for IdP-initiated SSO. Default: false
Always Prompt Re-authentication for SP-initiated SSO	Enabling this will require the user to re-authenticate, even if there is a valid session, for SP-initiated SSO. Note: Enabling this will require re-authentication for all SP-initiated SSO. For situational re-authentication, where it is up to SP to decide if re-authentication is required or not, set this as false. Default: false
Digital Signature Tab	
Note: Only specify the public key for signature verification of the authentication request.	
Keystore	The path to the signature keystore file in JKS format. The public key(s) in the keystore is/are for signature verification of the SAML Request received from the Service Provider.

General Tab	
Keystore Passphrase	The password of the keystore.
Active Key Alias	The active key for signature verification.
Active Key Certificate's Expiry Date	The expiry date of the active key certificate.
Log Expired Certificate	If enabled, an error log is added when an expired certificate from the keystore is used. This has no impact on the usage of the certificate, as the operation is still allowed to proceed even if the certificate has expired. Default: false
Key Rollover	
New Key Aliases	The list of key aliases for key rollover. Aliases are used in order from the top to the bottom of the list for key rollover. The current active alias must be before the alias that will be rolled over to in the list. If there is currently no existing value configured for the Active Key Alias, the first alias in the New Key Aliases list is used. Usage of aliases restarts from the top of the list once again after reaching the bottom of the list.
Coexisting Key Alias	The old key to be rolled over from in the key rollover event. The value of Active Key Alias before the rollover will be updated to this in the rollover event.
Enable Coexisting Key	When enabled, both the active key and coexisting key are used for signature verification.
Start Date of Coexisting Key Usage	Date to start the verification key rollover process. Upon reaching the specified date, the value of the Coexisting Key Alias attribute is updated to that of the Active Key Alias attribute. The use of the coexisting key is then enabled. The value of the Active Key Alias attribute is updated to the next alias in the New Key Aliases list. If signature verification using the updated active key public key fails, the coexisting key public key is used instead. Note: The Start Verification Key Rollover button must be clicked for the above to take effect.
Stop Date of Coexisting Key Usage	Date to complete the verification key rollover process. Upon reaching the specified date, the use of the coexisting key is disabled. Enable Coexisting Key is set to "false" and the system stops the use of the old key; only the active key public key will be used for signature verification.

General Tab	
	Note: The Complete Verification Key Rollover button must be clicked for the above to take effect.
XML Encryption Tab	
Note: Only specify the public key for encryption of the authentication response/assertion.	
Keystore	Path to the encryption keystore file in JKS format. The public key(s) in the keystore is/are for encrypting the assertion of the SAML Response generated by the Identity Provider.
Keystore Passphrase	The password of the keystore.
Active Key Alias	The active key for encryption.
Encryption Algorithm	The algorithm used for the SAML Response's assertion encryption. Options: ○ "AES128" ○ "AES256" Default: AES_128
Log Expired Certificate	If enabled, an error log is added when an expired certificate from the keystore is used. This has no impact on the usage of the certificate, as the operation is still allowed to proceed even if the certificate has expired. Default: false
Key Rollover	
New Key Aliases	The list of key aliases for key rollover. Aliases are used in order from the top to the bottom of the list for key rollover. The current active alias must be before the alias that will be rolled over to in the list. If there is currently no existing value configured for the Active Key Alias , the first alias in the New Key Aliases list is used. Usage of aliases restarts from the top of the list once again after reaching the bottom of the list.
Active Key Certificate's Expiry Date	The expiry date of the active key certificate.
Date to Perform Key Rollover	Date to perform the encryption key rollover process. The value of Active Key Alias is updated to the next key in the New Key Aliases list. The updated active key is used for encryption. Note: The Start Encryption Key Rollover button must be clicked for the above to take effect.

General Tab	
Policy	
Application Policy	The application policy that will be applied to the OAuth client. This allows the OAuth client to whitelist specific authentication or reauthentication methods for users. This setting may be required if: ○ Enable User Token Revocation with Security Events attribute has been set to "true" in the OAuth Authz Server. ○ CLP is required to enforce authentication using only whitelisted authentication mechanisms configured in the policy. Note: For more details on how to configure the policy, refer to the UAM Administration Guide - Application Policy Configuration.

Enable Authentication Context Processing

AccessMatrix UAM supports the use of authentication contexts for SAML web SSO. Authentication contexts provide information about how a user is authenticated (i.e. the authentication method(s) used). By specifying the expected authentication context(s) in the SP's request to the IdP, you can require users to be authenticated using those authentication method(s).



Note: You can find out more about the concept of authentication context processing in [UAM SAML Features - Authentication Context Processing](#).

When UAM acts as the SAML IdP, both SP-initiated SSO and IdP-initiated SSO are possible. The configurations steps differ slightly, depending on which flow is used:

SP-initiated SSO	For SP-initiated SSO, the SP will provide the expected authentication contexts in the request sent to AccessMatrix (acting as the IdP). You should verify that the lookup modules that will be used for authentication are configured with the same authentication contexts that will be expected. To do so: 1. Navigate to Administration Dashboard > Configuration > Authentication > Lookup Module and select the lookup module(s) that will be permitted to be used for the authentication of the user's identity. 2. In their corresponding configuration page(s), locate the Response Authentication Context Mapping attribute.
------------------	---

	3. Ensure that the expected authentication context(s) are configured in the respective login module(s). i Note: By default, an appropriate authentication context should already be set for each login module. If necessary, you may specify a different, more appropriate authentication context value. 4. Save the changes, if needed.
IdP-initiated SSO	For IdP-initiated SSO, instead of the SP generating a request, AccessMatrix (acting as the IdP) initiates the SSO flow. To obtain the required authentication contexts, it refers to the SP (using the "UAM as Identity Provider" implementation) configurations in AccessMatrix. To configure the expected authentication contexts, do the following: 1. Navigate to Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Service Provider and click on the previously created SAML SP. 2. Under the Response Authentication Context Mapping attribute, select the authentication context(s) to be required (CTRL + click to select multiple values). 3. Save the changes. Next, verify that the login modules that will be used for authentication are configured with the same authentication contexts that you previously specified. For example, if you required 2FA by specifying "rn:oasis:names:tc:SAML:2.0:ac:classes:PasswordProtectedTransport" and "urn:oasis:names:tc:SAML:2.0:ac:classes:MobileTwoFactorUnregistered", ensure that the login modules you intend to use for 2FA are each configured with the respective authentication context. To do so: 1. Navigate to Administration Dashboard > Configuration > Authentication > Login Module and select the login module(s) that will be permitted to be used for the authentication of the user's identity. 2. In their corresponding configuration page(s), locate the Response Authentication Context Mapping attribute. 3. Ensure that the authentication context(s) configured in the respective login module(s) are the same as those previously configured in the SP. i Note: By default, an appropriate authentication context should already be set for each login module. If necessary, you may specify a different, more appropriate authentication context value. 4. Save the changes, if needed.
	i Note: You can refer to the appendix B. Authentication Context Classes for further details on each of the authentication context classes. The table provides details on the type of authentication methods that correspond to each context class.

Roll Over Key Pair for Signature Verification/Encryption (SP)

AccessMatrix UAM supports both SAML signature verification and encryption key rollovers with zero downtime.

Signature Verification Key Rollover

For signature verification key rollovers, zero downtime is achieved through the use of a coexisting key, which is the new key that will be rolled over to at the completion stage.

The verification key rollover process involves two stages:

1. **Start Stage – Roll over new key and allow use of both new key and old key for signature verification**

The start stage is triggered by clicking the **Start Verification Key Rollover** button after the specified **Start Date of Coexisting Key Usage** is reached. This triggers the rollover to the new key and allows the use of both the new and old keys for signature verification; if validation using the updated active key fails, the new coexisting key is used instead.

2. **Completion Stage – Stop use of old key for signature verification**

The completion stage is triggered by clicking the **Complete Verification Key Rollover** button after the specified **Stop Date of Coexisting Key Usage** is reached. Doing so causes the system to stop using the old key for signature verification.

Follow the steps below to perform the key rollover:

1. Under the **Keystore** attribute, configure the path to the signing keystore file containing the current active key and new keys for key rollover.
 2. Under **New Key Aliases**, enter the aliases of the keys in the keystore. The current active key must be at the top of the list, followed by the next key(s) for rollover.
 3. Configure **Start Date of Coexisting Key Usage** to specify the date of the start stage of the key rollover.
 4. Click the **Start Verification Key Rollover** button after the **Start Date of Coexisting Key Usage** is reached to start the key rollover. The use of both the coexisting key (old key) and the new key is enabled.
 5. Configure **Stop Date of Coexisting Key Usage** to specify the date of the completion stage.
-

-
- Click the **Complete Verification Key Rollover** button after the **Stop Date of Coexisting Key Usage** to complete the key rollover process. **Enable Coexisting Key** is set to "false" and the system stops the use of the old key.

Encryption Key Rollover

The encryption key rollover process involves only one stage:

- Clicking the **Complete Encryption Key Rollover** button after the specified **Date to Perform Key Rollover** is reached causes the system to use the new active key for encryption.

Follow the steps below to perform the key rollover:

- Under the **Keystore** attribute, configure the path to the signing keystore file containing the current active key and new keys for key rollover.
- Under **New Key Aliases**, enter the aliases of the keys in the keystore. The current active key must be at the top of the list, followed by the next key(s) for rollover.
- Configure the **Date to Perform Key Rollover** to specify the date to roll over and activate the use of the new encryption key for SAML Response encryption.
- Click the **Start Encryption Key Rollover** button after the **Date to Perform Key Rollover** to perform the key rollover process.

Edit SAML Configuration

To navigate to the **SAML Configuration** page, select **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > SAML Configuration**. The **SAML Configuration** page can be edited by clicking the **Edit** button.

View SAML Configuration Page Attribute Description

 Note: ' * ' denotes a mandatory attribute.	
Attribute	Description
*Configuration Id	Unique Id of SAML configuration. (uneditable)
*Configuration Name	Name of SAML configuration, e.g. "SAMLConfiguration".
Description	Description of SAML configuration.

Attribute	Description
Version	Entry version of SAML configuration. (uneditable)
Implementation	Implementation type, e.g. "SAML Configuration". (uneditable)
Attributes Tab	
*Default Identity Provider	To denote the default Identity Provider referenced by REST API call where Identity Provider (idpId) is not specified.

4.2. Integration with SAML Service Providers

The following pages provide detailed instructions on how to integrate AccessMatrix UAM with SAML Service Providers:

- [UAM SAML with Office 365](#)
- [UAM SAML with Google](#)
- [UAM SAML with FortiGate SSL VPN](#)
- [UAM SAML with JIRA](#)

4.2.1. UAM SAML with Office 365

This section is about integrating AccessMatrix UAM as IdP with Office 365 as a Service Provider.

Prerequisite

- Office 365 subscription
- Windows Powershell ISE
- Microsoft Entra ID Module for Windows PowerShell
- To install, run the following commands one by one:
`install-module AzureAD`
`install-module MSOnline`
- An admin account with the Global Administrator role (or Company Administrator role). The steps require modifying Office 365 domain authentication setting. It is important that the admin account is not included by SSO enforcement. Otherwise, the admin will fail to login to Powershell once the SSO setting is enabled.
- Two domains registered under the Office 365 subscription. One domain is used for SSO, the other domain is used as default domain.



Note: Microsoft does not allow SSO to the default domain.

- For example, *i-sprint.com.my* domain will have SSO enabled, and *isprintpl.onmicrosoft.com* does not have SSO. This means, all users under *i-sprint.com.my* must log in via SAML, whereas users under *isprintpl.onmicrosoft.com* are not subjected to Federated SSO enforcement.

Configure Office 365

Reference: [Microsoft Entra ID: Use a SAML 2.0 Identity Provider \(IdP\) for Single Sign On](#) (external link).

1. Launch Windows PowerShell.
2. Login to Microsoft Online Service by typing the following command:
`Connect-MsolService`
3. The previous command will launch a pop-up window asking for login ID and password. Enter the Office 365 admin account with the Global Administrator role.
4. List all the domains under the subscription by typing the following command:

`Get-MsolDomain`

-
5. Run the following command to enable SSO in Office 365 and to configure SAML.

Code

```
Set-MsolDomainAuthentication
  -DomainName i-sprint.com.my
  -Authentication Federated
  -SigningCertificate
MIICQzCCAaygAwIBAgIETkx4kTANBgkqhkiG9w0BAQUFADBmMQswCQYDVQQGEwJTRzE
LMAkGA1UECBMCU0cx EjAQBgNV
BAcTCVNpbmdhcGyZTERMA8GA1UEChMIaS1TcHJpbnQxDDAKBgNVBAsTA0VORzEVMBMG
A1UEAxMMA5lzcHJpbnQyY29t
MB4XDTEyMDgxODAyMjc yOVoxDTIxMDgxNTAyMjc yOVowZjELMAkGA1UEBhMCU0cx CZA
JBgNVBAGTA1NHMRIwEAYDVQQH
Ew1TaW5nYXBvc mUxETAPBgNVBAoTCGktU3ByaW50MQwwCgYDVQQLEwNFTkcxFTATBgN
VBAMTDGktd3ByaW50LmNvbTCB
nzANBgkqhkiG9w0BAQEFAAOBjQAwgYkCgYEAiWCToelQW773XVlt3/yXAimMttAjY27
G9Sz8wJkTaCIQRGbxK4uDx8h9
eM8pY6vj/qD8sPUj//w3ak2opwhh/F2bp4AzyXwPOhZ92M2JQzti4mR/PCJxulLYwrZ
B9jloRbefLwpT/KLe27lfF64G
1ZJx5NK314WR6tQNX8W4yJ8CAwEAATANBgkqhkiG9w0BAQUFAAOBgQBYtwPp5V0Qael
wHWRG1PhJpHUWrsfzUgUtmQO/
aEOhXSvkACkeWXJ91Nd8Zpn+NkRYepnl0NGEePCs00U11N/1G+2J6tu1uZYwiESsFPm
wF7hU2JvGLdw74ACrgtn5Q6a5
y/XNRz1V4LddzaPVUR8fBS4mKrcb2lZl6ItlTX78Q==//
  -LogOffUri http://~/~/usoserver.i-sprint.demo:8080/am5/logout
  -PassiveLogOnUri http://~/~/usoserver.i-
sprint.demo:8080/am5/samlssso
  -IssuerUri i-sprint.com.m
  -FederationBrandName "i-Sprint Trial IDP"
  -PreferredAuthenticationProtocol SAMLp
```

- a. **DomainName** specifies the fully qualified domain name (FQDN) to update. It is set to the domain where the SSO is to be enabled.
 - b. **FederationBrandName** is a friendly name that may be used for display by Microsoft during the SSO process.
 - c. **IssuerUri** specifies the URL of the Identity Provider issuer. Microsoft requires the IssuerUri to be the same as the DomainName.
 - d. **PassiveLogOnUri** is the Identity Provider SSO URL that receives the SAML authentication request.
 - e. **SigningCertificate** is the Identity Provider certificate used to sign the SAML assertion.
 - f. **LogOffUri** specifies the Identity Provider logout URL, if required.
- Refer to [Set-MsolDomainAuthentication](#) for more attribute details.
6. If the `Set-MsolDomainAuthentication` command is not working due to an error regarding the default domain, then a domain needs to be set, other than the domain used for SSO, as the default domain.
-

For example, to set *isprintpl.onmicrosoft.com* as the default domain without enabling SSO for this domain, run the following command:

```
Set-MsolDomain -IsDefault -Name isprintpl.onmicrosoft.com
```

7. Confirm the federated SSO setting by running the following command:

```
Get-MsolFederationProperty -DomainName i-sprint.com.my
```

Example output:

Code

```
ActiveLogOnUri                :  
http://dummystsurl.microsoftonline.com/dummyurl  
DefaultInteractiveAuthenticationMethod :  
FederationBrandName          : i-Sprint Innovations Pte  
Ltd  
IssuerUri                     : i-sprint.com.my  
LogOffUri                     : http://usoserver.i-  
sprint.demo:8080/am5/logout  
MetadataExchangeUri          :  
NextSigningCertificate        :  
OpenIdConnectDiscoveryEndpoint :  
PassiveLogOnUri               : http://usoserver.i-  
sprint.demo:8080/am5/samlss  
SigningCertificate            :  
MIICQzCCAaygAwIBAgIETkx4kTANBgkqhkiG9w0BAQUFADBmMQswCQYDVQQGEwJTRzE  
LMAkGA1UECBMCU0cxEjAQBgNV  
BAcTCVNpbmdhcGyzTERMA8GA1UEChMIaS1TcHJpbnQxDDAKBgNVBAsTA0VORzEVMBMG  
A1UEAxMMaS1zcHJpbnQxY29t  
MB4XDTEyMDgxODAyMjc5OVoXDTIxMDgxNTAyMjc5OVowZjELMAkGA1UEBhMCU0cx  
CzA  
JBgNVBAgTAlNHMRIwEAYDVQQH  
Ew1TaW5nYXBvcnUxETAPBgNVBAoTCGktU3ByaW50MQwwCgYDVQQLEwNFTkcxFTATBgN  
VBAMTDGk3ByaW50LmNvbTCB  
nzANBgkqhkiG9w0BAQEFAAOBjQAwgYkCgYEAiWCToelQW773XVlt3/yXAimMttAjY27  
G9Ssz8wJkTaCIQRGbxK4uDx8h9  
eM8pY6vj/qD8sPUj//w3ak2opwhh/F2bp4AzyXwPOhZ92M2JQzti4mR/PCJxulLYwrZ  
B9jloRbefLwpT/KLe27lfF64G  
1ZJx5NK314WR6tQNX8W4yJ8CAwEAATANBgkqhkiG9w0BAQUFAAOBgQBBytwPp5V0Qael  
wHwG1PhJpHUWrSfzUgUtmQO/  
aEOhXSvkACkeWXJ91Nd8Zpn+NkRYepnl0NGEePCs00U1lN/1G+2J6tuluzYwiESsFpm  
wF7hU2JvGLdw74ACrgtn5Q6a5  
y/XNRz1V4LddzaPVUR8fBS4mKrcb21Z16It1TX78Q==  
SupportsMfa                   : False
```

Configure AccessMatrix UAM as IdP

In this integration, AccessMatrix UAM acts as the IdP and therefore there is a need to configure in Admin Console the attributes described in the [Identity Provider](#) section.



Note: The Identity Provider issuer ("Id used as issuer" attribute) must be the same with the Office 365's domain name that has the SSO feature.

After SAML IdP configuration, configure the **Default Identity Provider** attribute in [SAML Configuration](#) to point to the recently created SAML IdP, if not using the default identity provider.

Create a User Response

Microsoft Office 365 requires that the SAML Response has the following:

- Subject NameID where the value is Microsoft Entra ID user's Immutable ID
- "IDPEmail" SAML Attribute where the value is the user's User Principal Name (UPN) in Microsoft Entra ID/Microsoft 365. The UPN is in email address format.

This section provides the steps to create two User Responses (one is used for the Subject NameID containing the AD user's ImmutableID and the other is for IDPEmail Attribute).

Create a user response before configuring the Service Provider in the Admin Console.

1. Navigate to **Administration Dashboard > Configuration > Messaging & Response > User Response** and click the **Create** button.
2. Select "User Information Response Collector" from the Implementation list and click **Next**.
3. Enter a **User Response Id** (e.g. "Microsoft") and a **User Response Name** (e.g. Microsoft).
4. In the **User** tab, under the **Attributes** header, specify a name (e.g. "nameID") as the **Name (Claim Name)**.
5. Specify an attribute name (e.g. "microsoftImmutableId") that will return the Office 365 user's "ImmutableID" or "Object GUID" and click **Add**.



Notes:

- Office 365 user's "ImmutableID" may not be set. It can be set using the powershell command Set-MsolUser.
- Ensure that the "microsoftImmutableId" custom attribute is created and provided with Office 365 user's Immutable ID value in the User's page.

6. Click **Save**. A new User Response is created.
 7. Perform steps 1 and 2 to create another user response.
 8. Enter a **User Response Id** (e.g. "microsoftEmail") and a **User Response Name** (e.g. "microsoftEmail").
-

9. In the **User** tab, under the **Attributes** header, specify a name (e.g. "IdPEmail") as the **Name (Claim Name)**.
10. Select "email" as the attribute name that will return the Office 365's User Principal Name (UPN) and click **Add**.



Note: Ensure that the "email" custom attribute is created and provided with Office 365's User Principal Name (UPN) in the Users page.

11. Click **Save**. A new User Response is created.

Now, proceed to configure the SAML Service Provider.

Configure Microsoft SAML SP in AccessMatrix UAM

1. Obtain the metadata from Microsoft prior to the service provider configuration. **Metadata** is a set of data that describes and gives information about other data. Microsoft provides two global metadata: one for China, and the other one is for Global (excluding China). Below is an example of global metadata:

Microsoft Metadata (Global)

Code

```
<?xml version="1.0" encoding="utf-8"?>
<EntityDescriptor ID="_0c0d1ca7-7292-4bc6-801c-f880f6098f4e"
    entityID="urn:federation:MicrosoftOnline"
    xmlns="urn:oasis:names:tc:SAML:2.0:metadata"
    xmlns:alg="urn:oasis:names:tc:SAML:metadata:algsupport">
    <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
        <SignedInfo>
            <CanonicalizationMethod
Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" />
            <SignatureMethod
Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1" />
            <Reference URI="#_0c0d1ca7-7292-4bc6-801c-f880f6098f4e">
                <Transforms>
                    <Transform
Algorithm="http://www.w3.org/2000/09/xmldsig#enveloped-signature" />
                    <Transform Algorithm="http://www.w3.org/2001/10/xml-exc-
c14n#" />
                </Transforms>
                <DigestMethod
Algorithm="http://www.w3.org/2000/09/xmldsig#sha1" />
                <DigestValue>lmuywskSIZK9HjyNuvYE+Y2vtNU=</DigestValue>
            </Reference>
        </SignedInfo>
```

Microsoft Metadata (Global)

```
<SignatureValue>WbCHKG4bcAcRzKQIdNIuVQqdHgIpwpgh8RP5frAvf7+SZIxN3JIMW6FyX+Y
yQoPO8RV9YcBL8uTWFFD2xJlTafogDFyFE8gnzhZAwSppj4DO60v/vTxnjR3gEucdaJIfqfDIAaO
RmgSLurHdlGuhq/LZCjZ3d6lsnDVlpHbSjt/vDMlokfBk009RqIsh8UozdXaqrY8isNKjD3lUtAN
YPYTiJGiK9LeJd+UkzTjbluWxRmt4qaxWp8d92DeD/do/99cKY8Xy5uFsM2qXDHnQm6IH17HWJWM
P7oQN53NtnVDq+8PqabF2q6Ai8s/NMJUvoYN/ELCIUZ57pD+1A74WQ==</SignatureValue>
  <KeyInfo>
    <X509Data>

<X509Certificate>MIIDYDCCAkigAwIBAgIJALLJPAyv2sjMA0GCSqGSIb3DQEBAQ
QUAMCkxJzAlBgNVBAMTHkxpdmUgSUQgU1RTIFNpZ25pbmcgUHVibGljIETleTAeFw0xNDA3MTgxO
TUzNDBaFw0xOTA3MTcxOTUz
NDBaMCKxJzAlBgNVBAMTHkxpdmUgSUQgU1RTIFNpZ25pbmcgUHVibGljIETleTCCASIwDQYJKoZI
hvcNAQEBBQADggEPADCCAQo
CggEBANYDKgByFZdqtTnnpF4IfIp4i2XLg2rLIo+mu4DmW9gRLlBJCnc7YESUxpKzuFYaANd8fWs
DigJZTXbhOQApSpw4xxFnor
2vJlzm94LtqjcVEXTjUml5gAIS4pwuOU3ZfO/0eTG0gDYp4a0L/mzzTRsnwe/8WMPiE75Bq2zAyA
Z9aePv13QX7cXYLPfeK4QTg
K3B5lwe1wWu3y5oQidjcSok8Fr80xzuCYuOa+ZUK3JibpLLCrT4uwiqf+KREDSdc4bPPlq0PWI4
sQr1tha8yypRSvOH+/Mxcfs
RSnl6Uc+gm8nVEEWWIu4hhu6NIfG91mMUqJuzkgLCi6Gov6JS8UCAwEAAaOBijCBhzAdBgNVHQ4E
FgQUNQoq7sI3R8rde4sQs6n
GEbJm3LcwWQYDVR0jBFIwUIAUnQoq7sI3R8rde4sQs6nGEbJm3LehLaQrMCkxJzAlBgNVBAMTHkx
pdmUgSUQgU1RTIFNpZ25pbm
cgUHVibGljIETleYIJALLJPAyv2sjMAsGA1UdDwQEAwIBxjANBgkqhkiG9w0BAQUFAAOCAQEAf4
jaNhKzRG3k+52WoM9nnISP7
rlWIeWwH6EQGUlF6ozSP/03gYMAAdqpDhw5zNwKzi7TQVbDC0pgq/tqzHv6JEI0R4B6h7/TJ1pYP
xdvIFQrE27RHESlth/m+5Uk
VnayLqRD3/fi4zf4aEpxSDZ73MCR5LanPGqvlAMz29AL3glynj+eu7xMfFsm/8+qJaCXut5/30e
eLEe+PYika/PhEwp+qkDQWP
vdAwEghuUaFvtKAgDZierjpGzHZnYkXTTDTHVelip7tsAJH5qK3qdcv3UGPyZrjC/lietJcAcnwV
oZQ93v2ieGfcKKN+PFN9M59
/BkPo62HPoGNNx2ZDQaQ==
    </X509Certificate>
  </X509Data>
</KeyInfo>
</Signature>
<Extensions>
  <alg:DigestMethod
Algorithm="http://www.w3.org/2000/09/xmldsig#sha1"/>
  <alg:SigningMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-
sha1"/>
</Extensions>
  <SPSSODescriptor
protocolSupportEnumeration="urn:oasis:names:tc:SAML:2.0:protocol"
WantAssertionsSigned="true">
  <KeyDescriptor use="signing">
    <ds:KeyInfo xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
      <ds:X509Data>

<ds:X509Certificate>MIIDYDCCAkigAwIBAgIJALLJPAyv2sjMA0GCSqGSIb3DQEBAQ
QUAMCkxJzAlBgNV
BAMTHkxpdmUgSUQgU1RTIFNpZ25pbmcgUHVibGljIETleTAeFw0xNDA3MTgxOTUz
NDBaFw0xOTA3MTcxOTUzNDBaMCKxJzAlBgNVBAMTHkxpdmUgSUQgU1RTIFNpZ25p
bmcgUHVibGljIETleTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBANYD
KgByFZdqtTnnpF4IfIp4i2XLg2rLIo+mu4DmW9gRLlBJCnc7YESUxpKzuFYaANd8
fWsDigJZTXbhOQApSpw4xxFnor2vJlzm94LtqjcVEXTjUml5gAIS4pwuOU3ZfO/0
```


Microsoft Metadata (Global)

```
eTG0gDYp4a0L/mzzTRsnwe/8WMPiE75Bq2zAyAZ9aePv13QX7cXYLPfeK4QTgK3B
5lwe1wWu3y5oQidjcSok8Fr80xzuCYuOa+ZUK3JibpLLCrT4uwiqf+KREDSdc4b
PPlq0PWI4sQr1tha8yypRSvOH+/MxcfSRsNl6Uc+gm8nVEEWWIu4hhu6NI fG91mM
UqJuzkgLCi6Gov6JS8UCAwEAAaOBijCBhzAdBgNVHQ4EFgQUnQoq7sI3R8rde4sQ
s6nGEbJm3LcwWQYDVR0jBFIwUIAUnQoq7sI3R8rde4sQs6nGEbJm3LehLaQrMCkx
JzAlBgNVBAMTHkxpdmUgSUQgU1RTIFNpZ25pbmcgUHVibGljIETleYIJALLJPAYv
f2sjMAsGAlUdDwQEAwIBxjANBgkqhkiG9w0BAQUFAAOCAQEAF4jaNhKzRG3k+52W
om9nnISP7rlWIEwW6EQGUlF6ozSP/03gYMAAdqpdhww5zNwKzi7TQVbDC0pgq/tq
zHv6JEI0R4B6h7/TJlpYPxdvIFQrE27RHESltH/m+5UkVnayLqRD3/fi4zf4aEpx
SDZ73MCR5LanPGqvlAMz29AL3glynj+eu7xMfFsM/8+qJaCXuxT5/30eeLEe+PYi
kA/PhEwp+qkDQWPvdAwEghuUaFvtKAqDZierjpGzHZnYkXTTDTHVelip7tsAJH5q
K3qdcv3UGPyZrjC/lietJcAcnwVoZQ93v2ieGfcKKN+PFN9M59/BkPo62HPoGNNx
2ZDQaQ==
</ds:X509Certificate>
</ds:X509Data>
</ds:KeyInfo>
</KeyDescriptor>
<KeyDescriptor use="signing">
  <ds:KeyInfo xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
    <ds:X509Data>
<ds:X509Certificate>MIIDYDCCAkigAwIBAgIJAKLDsqkylLefMA0GCSqGSIb3DQEBBQUAMCkx
JzAlBgNV
BAMTHkxpdmUgSUQgU1RTIFNpZ25pbmcgUHVibGljIETleTAeFw0xNDEwMTAxODE2
MTNafW0xOTEwMDkxODE2MTNafMCkxJzAlBgNVBAMTHkxpdmUgSUQgU1RTIFNpZ25p
bmcgUHVibGljIETleTCCASIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAM7A
3m6uvOxExX+NlB1hnflar8DJj597wY3qyh/FX406rKvU2leAfINmBWcjEFApCKi9
p5uIaZpNlDpPQ+R3BaZx+4NhHbOMpewlPiZHL6llwbulzurffUPhtzQNHAVzOBk
ZsOgN9BD/hOleU//d+IXz08ateUb3Ip2vyaodilYQDDi5M9yOhanv1c01Usjo2xt
LfiK+TVygu+8bo+/8JHGPRy6pngnhg970DRBDkVrKzozlrnmMesdSrtuCnsgyRbE
XckxaQ8S2nDYyFqBI0PkcBW8+0akdFWW58Os5cGbPFfHi6vtZCR5pWw5pnqtuoip
rdk9jglaxT3vwu+RVdcCAwEAAaOBijCBhzAdBgNVHQ4EFgQUBjNylGJBvkAY/4yI
IoD00R6p5hIwWQYDVR0jBFIwUIAUBjNylGJBvkAY/4yIIoD00R6p5hKhLaQrMCkx
JzAlBgNVBAMTHkxpdmUgSUQgU1RTIFNpZ25pbmcgUHVibGljIETleYIJAKLDsqky
lLefMAsGAlUdDwQEAwIBxjANBgkqhkiG9w0BAQUFAAOCAQEAGZU1J3zzJvy10Ld
tV3NTYHlBVHm3Fty17xqW9Ui8GE8sEWEuDH6eURNpNpd+gAGC6Tp+k+cU1LlPw
Xm7BAATJ/2DjY8tzRc6r6EneQWRkIa8xpbnknXvUml6iFgo2ofOWLaFk6XpQ64MA
O35wv9XEARNabJ9wJSRSevUigAx2U2GvaorV5PgqHImiKTSrL0K6j8B4OqXWUqP0
KGf7pCdGlrlq2XEl95N2zj8n/scvA9JasImztsVlZ+WxeF+OAMvWQQFc54gC6lwWc
8kno8vPn3lwxVktU0o9wcHnOhNi2hzVDV85sz7P9dOZYF73uyluLshdjCcwlmQ2l
A9OV9w==
</ds:X509Certificate>
</ds:X509Data>
</ds:KeyInfo>
</KeyDescriptor>
<SingleLogoutService
Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"

Location="https://login.microsoftonline.com/login.srf"/>
  <NameIDFormat>urn:oasis:names:tc:SAML:1.1:nameid-
format:emailAddress</NameIDFormat>
  <NameIDFormat>urn:mace:shibboleth:1.0:nameIdentifier</NameIDFormat>
  <NameIDFormat>urn:oasis:names:tc:SAML:1.1:nameid-
format:unspecified</NameIDFormat>
  <NameIDFormat>urn:oasis:names:tc:SAML:2.0:nameid-
format:transient</NameIDFormat>
  <NameIDFormat>urn:oasis:names:tc:SAML:2.0:nameid-
```

Microsoft Metadata (Global)

```
format:persistent</NameIDFormat>
  <AssertionConsumerService isDefault="true"
    index="0"

Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"

Location="https://login.microsoftonline.com/login.srf"/>
  <AssertionConsumerService index="1"

Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST-SimpleSign"

Location="https://login.microsoftonline.com/login.srf"/>
  <!-- PAOS functionality is NOT supported by this service. The
binding is only included to ease setup and integration with Shibboleth ECP -
->
    <AssertionConsumerService index="2"

Binding="urn:oasis:names:tc:SAML:2.0:bindings:PAOS"

Location="https://login.microsoftonline.com/login.srf"/>
  </SPSSODescriptor>
</EntityDescriptor>
```

2. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Service Provider** and click **Create** button.
3. Select the "UAM as Identity Provider" implementation and click **Next**.
4. Enter a **Service Provider Id** (e.g. "office365") and a **Service Provider Name** (e.g. "office365").
5. Specify "urn:federation:MicrosoftOnline" as the value for the **Id used as issuer** attribute.
6. In the **General** tab, under the **Assertion** header:
 - a. Specify "https://login.microsoftonline.com/login.srf" as the value for **Assertion Consumer Service URLs** attribute.
7. Under the **Name Id** header:
 - a. Select "Persistent" as the value for the **Format** attribute.
 - b. Select "User Response" as the value for the **Value** attribute.
 - c. Specify the previously created User Response (i.e. "Microsoft") as the value for the **User Response to return as NameId value** attribute.
8. Under the **Attributes** header:
 - a. Specify the previously created User Response (i.e. "microsoftEmail") as the value for the **Attributes to return in the assertion** attribute.

9. In the **Web Browser SSO** tab:

a. Specify

"https://login.microsoftonline.com/login.srf?wa=wsignin1.0&whr=<DOMAIN_NAME>" as the value for the **URL to initiate SSO** attribute.



Note: Change the <DOMAIN_NAME> to the domain name configured for SSO.

10. Save the changes. A new Service Provider is created.

4.2.2. UAM SAML with Google

This section is about integrating AccessMatrix UAM as IdP with Google as a Service Provider.

Prerequisite

- A Google admin account for Google Workspace or Google Apps

Configure AccessMatrix UAM as IdP

In this integration, AccessMatrix UAM acts as the IdP and therefore there is a need to configure in Admin Console the attributes described in the [Identity Provider](#) section.

The IdP's certificate used for signing is required as part of the steps to configure Google. If you don't have the certificate file, it can be extracted from the Keystore or from the SAML IdP metadata. To extract IdP certificate from the SAML IdP metadata file, you may click [here](#) to find out more.

After SAML IdP configuration, configure the **Default Identity Provider** attribute in [SAML Configuration](#) to point to the recently created SAML IdP, if not using the default identity provider.

Configure Google

1. In the Google Admin console (at admin.google.com), go to **Security**.
 2. Click **Set up single sign-on (SSO) with a third party IdP**.
 3. Click **Add SSO profile**.
 4. Check the **Set up SSO with third party identity provider** box.
 5. Enter the following URLs to your third-party IdP:
 - a. **Sign-in page URL:** The page where users sign in to your system and Google Workspace, e.g. *https://clp.i-sprint.com/clp/samlssso*.
 - b. **Sign-out page URL:** The page where users are redirected to after signing off, e.g. *https://clp.i-sprint.com/clp/logout*.
 6. Upload the certificate that was downloaded and prepared in the [previous](#) section.
 7. Check **Use a domain-specific issuer** so that Google will send the issuer specific to your domain, "google.com/a/<DOMAIN_NAME>", where <DOMAIN_NAME> is replaced with your actual domain name.
-



Note: You need this <DOMAIN_NAME> value for configuring SAML Service Provider in AccessMatrix server.

8. Click **Save**.

Configure Google SP in AccessMatrix UAM

The following sections provide the information and preparations required for configuring Admin Console's Service Provider.

Information Provided by Google

- **Subject NameId**

Google requires the SAML Response to have the end user's email address as the value of the Subject NameID.

- **ACS URL**

The Assertion Consumer Service (ACS) URL, tells the IdP where to redirect an authenticated user after sign-in. An ACS URL takes the following form:

https://www.google.com/a/<DOMAIN_NAME>/acs



Note: If your organization restricts access to *www.google.com*, you may contact your organization's support team for an alternate ACS URL and mention the [Service provider SSO setup](#) article.

Preparation

Prior to creation of Google application as a SAML Service Provider in AccessMatrix UAM, you may want to read [User Entitlement and Identity Mapping](#) to evaluate the following:

- Identity mapping strategy between AccessMatrix UAM user identity and Google user's identity.
- If all users are allowed to access the Google apps or Workspace.

Create SAML Service Provider

Perform the following steps to create a new SAML Service Provider instance with the required configuration:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Service Provider** and click the **Create** button.
 2. Select the "UAM as Identity Provider" implementation.
-

-
3. Enter a **Service Provider Id** (e.g. "Google") and a **Service Provider Name** (e.g. "Google").
 4. Specify "google.com/a/<DOMAIN_NAME>", which you can obtain from the [previous](#) section, as the value for **Id used as issuer** attribute.
 5. In the **General** tab, under the **Assertion** header:
 - a. Specify "https://www.google.com/a/<DOMAIN_NAME>/acs" as the value for the **Assertion Consumer Service URLs** attribute, where <DOMAIN_NAME> is replaced with your actual primary Google Workspace domain name.
 6. Under the **Name Id** header:
 - a. Select "Email" as the value for the **Value** attribute.
 7. In the **Web Browser SSO** tab:
 - a. If USO SSO Portal is used, specify "https://<GoogleAppURL>/a/<DOMAIN_NAME>" as the value for the **URL to initiate SSO** attribute. This allows USO SSO Portal's user to login to the corresponding Google App via SAML (AccessMatrix UAM as the SAML IdP).
 - i. Replace <DOMAIN_NAME> with your actual primary Google Workspace domain name.
 - ii. Replace <Google_App_URL> with the intended Google app URL, e.g. "chat.google.com", "drive.google.com", "mail.google.com", etc.
 8. Save the changes.
 9. A new Service Provider is created.
-

4.2.3. UAM SAML with FortiGate SSL VPN

This section provides instructions on integrating AccessMatrix UAM as the SAML Identity Provider with Fortinet FortiGate SSL VPN as the SAML Service Provider. This implementation uses the Common Login Page (CLP) as the IdP front end and the UAM Server as the IdP back end. The FortiGate SSL VPN authentication requires the use of both the Fortinet FortiGate server and FortiClient VPN.



Note: This guide assumes you have already set up a functional FortiGate server; it does not provide instructions on how to do so.

Prerequisites

- The user must have successfully performed SSL VPN authentication to the FortiGate server.
- Download the FortiClient VPN software.
- Ensure the IdP server (AccessMatrix UAM server) has a public IP address.
- Ensure the IdP server has a signed SSL certificate.

Configure AccessMatrix

Configure AccessMatrix UAM as IdP

1. Log in to the Admin Console and navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Identity Provider**.
2. Click the **Create** button and select "UAM as Identity Provider" from the **Implementation** dropdown list. Click **Next**.
3. Fill in the **Identity Provider Id** and **Identity Provider Name** fields with appropriate values (e.g. "UAMIdP" and "AccessMatrix UAM as IdP").
4. Fill in the **Id used as issuer** field with the domain name of the IdP server (e.g. "https://idpserver.com").
5. In the **Web Browser SSO** tab, add the IdP login page URL under the **Single Sign-On Service URLs** ;attribute. Check the **HTTP Redirect** radio button, enter the URL into the text box (e.g. "https://idpserver.com/clp/samlssso") and click **Add**. Repeat this for **HTTP POST**.



Note: This URL depends on your configured IdP front end - refer to [Set Up Identity Provider Front End](#) for more information. If you are using the AccessMatrix Common Login Page (CLP) as your IdP front end, the URL will be <IdP domain>/clp/samlssso.

-
6. In the **Digital Signature** tab, configure the following attributes:
 - a. **Keystore:** Enter the path to the IdP SSL certificate public key (e.g. "C:/certificate.p12").
 - b. **Keystore Passphrase:** Enter the certificate password.
 - c. **New Key Aliases:** Add the alias of the public key by entering the alias into the empty field and clicking the **Add** button.
 7. Save the changes.

Create User Response

1. Log in to the Admin Console and navigate to **Administration Dashboard > Configuration > Messaging & Response > User Response**.
2. Click the **Create** button and then click **Next**.
3. Fill in the **User Response Id** and **User Response Name** fields as appropriate (e.g. "FortiGateResponse" and "FortiGate User Response").
4. Under **Fields**, add the following:

Name (Claim Name)	Attribute Name
"group-name"	"GroupMembership"
"user-name"	"UserName"

5. Save the changes.

Configure FortiGate as SP in AccessMatrix UAM

Log in to the Admin Console and navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Service Provider**. Perform the following steps:

1. Click the **Create** button and select "UAM as Identity Provider" from the **Implementation** dropdown list. Click **Next**.
 2. Fill in the **Service Provider Id** and **Service Provider Name** fields with appropriate values (e.g. "FortiGateSP" and "FortiGate SSL VPN as SP").
-

-
3. Fill in the **Id used as issuer** field with the value "<URL of SP server (FortiGate server)>/remote/metadata".

For example, "https://ssl.spserver.com:1443/remote/saml/metadata".



Important Note: Ensure that there are no other Service Providers using the same **Id used as issuer** in AccessMatrix. If there are multiple SPs with the same **Id used as issuer**, you will encounter errors while attempting to configure the FortiGate server.

4. Under the **Assertion** header, configure the value of **Assertion Consumer Service URLs** to "<URL of SP server (FortiGate server)>/remote/saml/login".
For example, "https://ssl.spserver.com:1443/remote/saml/login".
5. Under the **Name Id** header, configure the following attributes:
 - a. **Format:** "Unspecified"
 - b. **Value:** "Id"
6. Under the **Attributes** header, set the value of **Attributes to return in assertion** to the user response created in the [previous section](#) (e.g. "FortiGateResponse")
7. Save the changes.

Create Group

Log in to the Admin Console and navigate to **Operation Dashboard > User & Segment > Group**.

Perform the following steps:

1. Click the **Create** button.
2. Fill in the **Group Id** and **Group Name** fields with appropriate values (e.g. "FortiGateSSLVPNGroup" and "FortiGate SSL VPN Group").
3. Save the changes.

Create User

Log in to the Admin Console and navigate to **Operation Dashboard > User & Segment > User**. Perform the following steps:

1. Click the **Create** button. The **Create User** page is displayed.
 2. Fill in the **User Id** field with the user's email. Fill in **User Name** field with an appropriate value.
-

Note: Alternatively, click the **Search** button on the **User** page and click on the desired user. Ensure that the **User Id** value has been set to the user's email.

- Under the **Login Accounts** tab, assign a login realm to the user (e.g. "Default Password Basic (40151)") by clicking the **Add** button and selecting the realm in the dropdown list.
- Under the **Membership** tab, assign the user to the group created in the [previous section](#) (e.g. "FortiGateSSLVPNGroup") by ticking the corresponding checkbox.
- Save the changes.

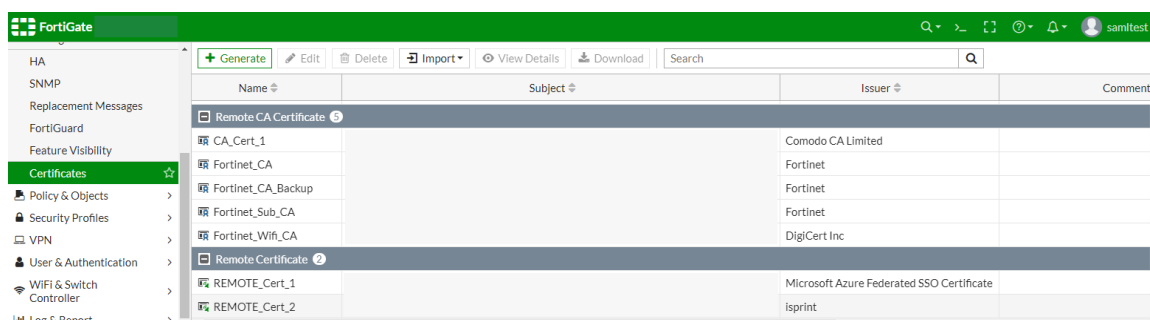
Configure FortiGate Server

Import IdP Certificate

- Log in to the FortiGate WebUI and navigate to **System > Certificates** on the left pane.
- Click on the **Import** button in the menu bar and select "Remote Certificate" from the dropdown list.
- Click **Upload** and browse to the IdP certificate location.

Note: Ensure that the certificate is in Base64 format.

- Click **OK**. The newly uploaded certificate is displayed in the **Remote Certificate** list on the **Certificates** page (e.g. "REMOTE_Cert_2").



Create SAML IdP

The following configurations rely on the FortiGate Command Line Interface (CLI). Ensure that your local machine is connected to the FortiGate CLI before performing the steps below.

Note: Refer to the [FortiGate documentation](#) for more details on establishing a connection to the FortiGate CLI.

In the FortiGate CLI, create a SAML IdP using the following commands:

Create SAML IdP
<pre> conf user saml edit "<IdP Name used in FortiGate>" set entity-id "<SP Issuer Id>" set single-sign-on-url "<SP ACS URL>" set single-logout-url "<SP Logout URL>" set idp-entity-id "<IdP Issuer Id>" set idp-single-sign-on-url "<IdP SSO Services URL>" set idp-single-logout-url "<IdP Logout URL>" set idp-cert "<Imported IdP Certificate>" set user-name "<SAML Response Attribute Name for User Name>" set group-name "<SAML Response Attribute Name for Group Name>" next end </pre>

Replace the values enclosed in "<>" as appropriate. For example:

Create SAML IdP Sample
<pre> conf user saml edit "AM IdP" set entity-id "https://ssl.spserver.com:1443/remote/saml/metadata" set single-sign-on-url "https://ssl.spserver.com:1443/remote/saml/login" set single-logout-url "https://ssl.spserver.com:1443/remote/saml/logout" set idp-entity-id "https://idpserver.com" set idp-single-sign-on-url "https://idpserver.com/clp/samlssso" set idp-single-logout-url "https://idpserver.com/clp/logout" set idp-cert "REMOTE_Cert_2" set user-name "user-name" set group-name "group-name" next end </pre>

The table below provides a detailed explanation of each of the values:

Value	Description	Example
<IdP Name used in FortiGate>	Any appropriate name for the IdP currently being created in FortiGate.	AM-IdP
<SP Issuer Id>	The SP issuer Id. This is the same value specified for the Id used as Issuer attribute in the	https://ssl.spserver.com:1443/remote/saml/metadata

Value	Description	Example
	Configure FortiGate as SP in AccessMatrix UAM section.	
<SP ACS URL>	The SP Assertion Consumer Service (ACS) URL. This is the same value specified for the Assertion Consumer Service URLs attribute in the Configure FortiGate as SP in AccessMatrix UAM section.	https://ssl.spserver.com:1443/remote/saml/login
<SP Logout URL>	The SP logout URL. It should follow the format "https://<URL of SP server (FortiGate server)>/remote/saml/logout".	https://ssl.spserver.com:1443/remote/saml/logout
<IdP Issuer Id>	The IdP issuer Id. This is the same value specified for the Id used as issuer attribute in the Configure AccessMatrix UAM as IdP section.	https://idpserver.com
<IdP SSO Services URL>	The IdP Single Sign-On (SSO) Services URL. This is the same value specified for the Single Sign-On Services URLs attribute in the Configure AccessMatrix UAM as IdP section. The URL depends on your configured IdP front end - refer to Set Up Identity Provider Front End for more information.	https://idpserver.com/clp/samlssso Note: This example assumes the AccessMatrix Common Login Page (CLP) is used as the IdP front end.

Value	Description	Example
<IdP Logout URL>	The IdP logout URL. This URL depends on your IdP front end - refer to Set Up Identity Provider Front End for more information.	https://idpserver.com/clp/logout Note: This example assumes the AccessMatrix Common Login Page (CLP) is used as the IdP front end.
<Imported IdP Certificate>	The name of the IdP certificate imported in the previous section .	REMOTE_Cert_2
<SAML Response Attribute Name for User Name>	The SAML Response attribute name for the user name attribute, as configured in the AccessMatrix Admin Console. This is the same value specified under Name (Claim Name) for "UserName" in the Create User Response section.	user-name
<SAML Response Attribute Name for Group Name>	The SAML Response attribute name for the group name attribute, as configured in the AccessMatrix Admin Console. This is the same value specified under Name (Claim Name) for "GroupMembership" in the Create User Response section.	group-name

Create SAML Group

In the FortiGate CLI, create a SAML group using the following commands:

Create SAML Group
<pre> conf user group edit "<User Group Name used in FortiGate>" set member "<IdP Name used in FortiGate>" config match </pre>

Create SAML Group
<pre> edit 1 set server-name "<IdP Name in used FortiGate>" set group-name "<AccessMatrix Group Id>" next end next end </pre>

Replace the values enclosed in "<>" as appropriate. For example:

Create SAML Group Sample
<pre> conf user group edit "FortiGateSAMLGroup" set member "AM-IdP" config match edit 1 set server-name "AM-IdP" set group-name "FortiGateSSLVPNGroup" next end next end </pre>

The table below provides a detailed explanation of each of the values:

Value	Description	Example
<User Group Name used in FortiGate>	Any appropriate name for the user group currently being created in FortiGate.	FortiGateSAMLGroup
<IdP Name used in FortiGate>	The name of the IdP previously created in FortiGate. This is the same value represented by <IdP Name used in FortiGate> in the previous section .	AM-IdP
<AccessMatrix Group Id>	The group Id for the group previously configured within the AccessMatrix Admin Console. This is the same value specified for Group Id in the Create Group section.	FortiGateSSLVPNGroup

Create Firewall Policy for SSL VPN

-
1. Log in to the FortiGate WebUI and navigate to **Policy & Object > Firewall Policy** on the left pane.
 2. Click the **Create New** button in the menu bar.
 3. Enter a suitable name under **Name** (e.g. "REMOTE_ACCESS_VPN").
 4. For **Source Address**, click the **+** button and navigate to the **User** tab. Under **USER GROUP**, select the SAML group that was created in the [previous section](#) (e.g. "FortiGateSAMLGroup").
 5. Configure the remaining fields as appropriate, based on your FortiGate deployment. Refer to [FortiNet's documentation](#) for more information on FortiGate firewall policy configuration.
 6. Click **OK**. The newly created policy is displayed in the **Firewall Policy** page:

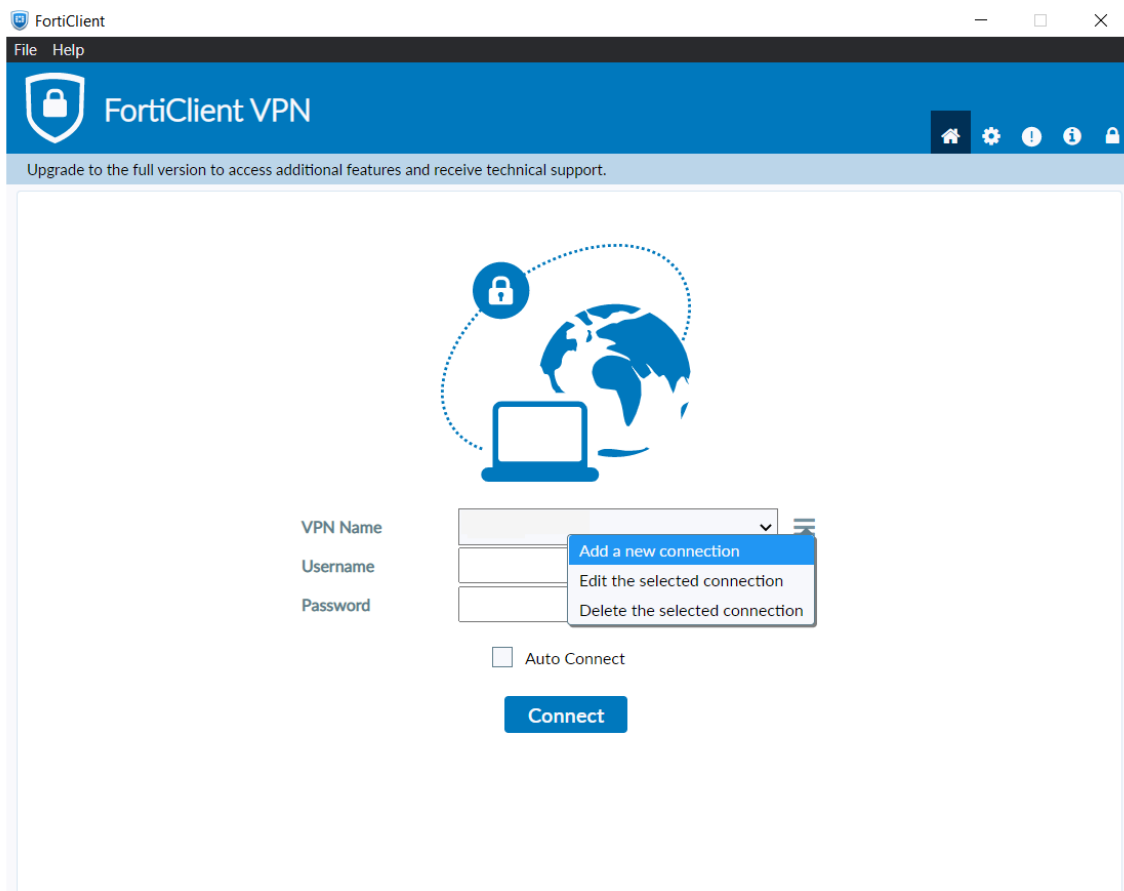
name	source interface	destination interface	source address	Destination address	schedule	Serve	action	NAT	security configuration	traffic log
REMOTE_ACCESS_VPN...	SSL-VPN tunnel interface (sslroot)	Intranet (LCAP Port11...	FortiGateSAMLGroup	Network_... Network_...	always	ALL	✓ acce...	disab...	no-inspection	all

Configure FortiClient VPN

Set Up FortiClient

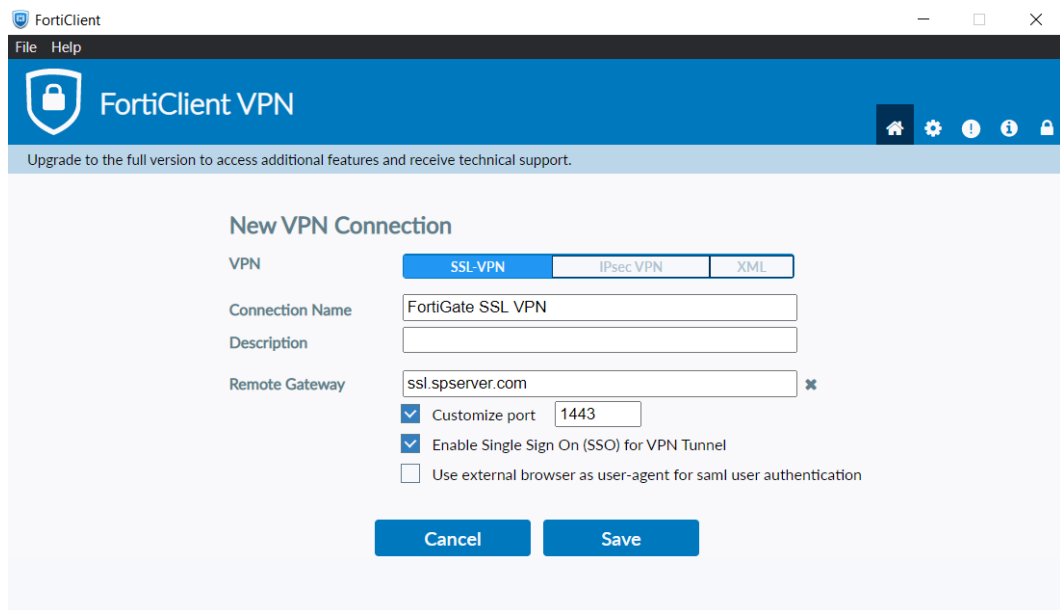
1. Launch FortiClient VPN.
-

2. Click on the menu icon and select "Add a new connection" from the dropdown list.



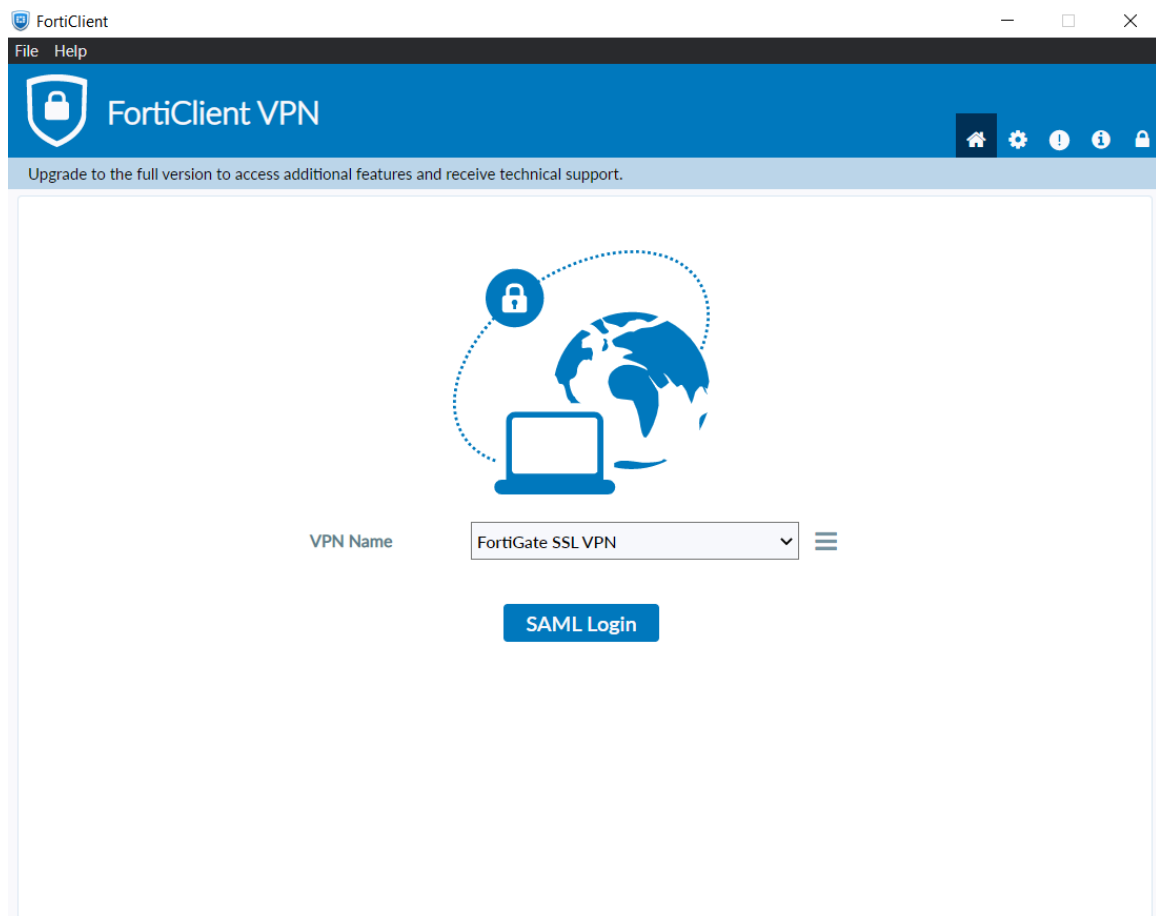
3. Fill in the **Connection Name** field as appropriate (e.g. "FortiGate SSL VPN").
4. Configure the following:
 - a. **Remote Gateway:** Enter the domain of the SP server (e.g. "ssl.spserver.com").
 - b. **Customize port:** Enter the appropriate port number for the SP server and tick the checkbox.

- c. **Enable Single Sign On (SSO) for VPN Tunnel:** Tick the checkbox.



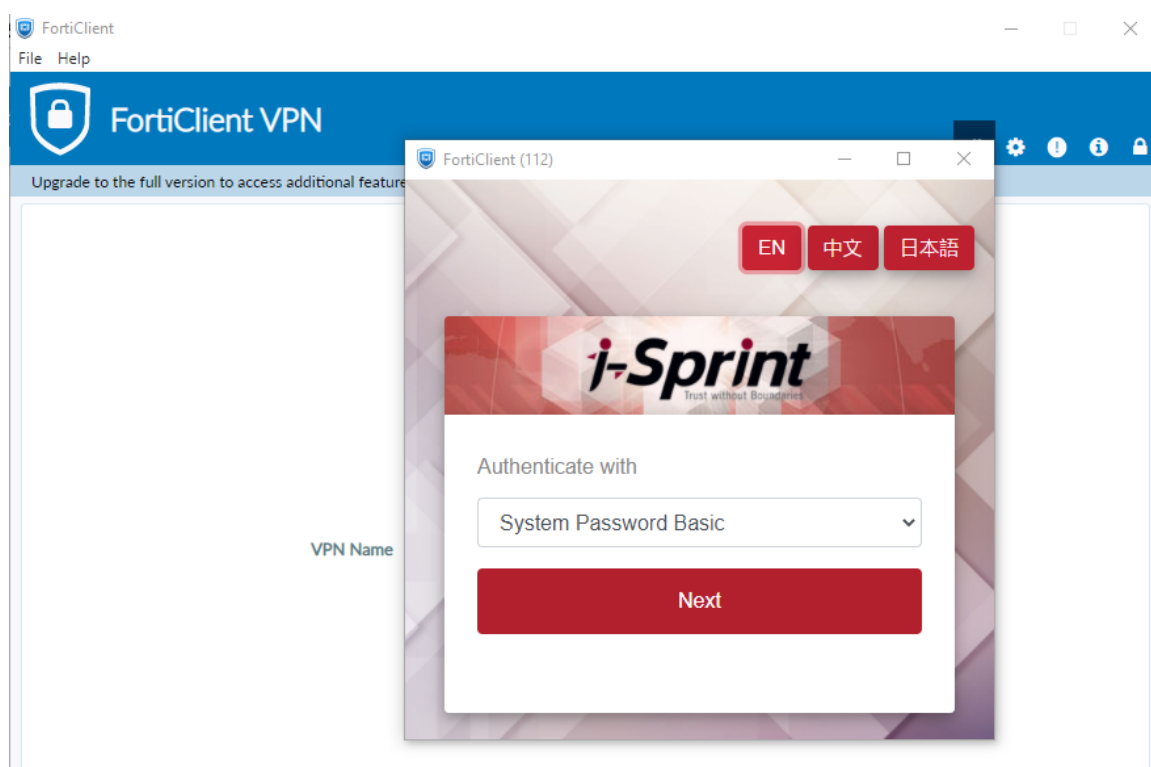
The screenshot shows the 'New VPN Connection' dialog box in the FortiClient application. The 'VPN' tab is selected, showing three options: 'SSL-VPN' (selected), 'IPsec VPN', and 'XML'. The 'Connection Name' field is filled with 'FortiGate SSL VPN'. The 'Remote Gateway' field is filled with 'ssl.spserver.com'. The 'Customize port' checkbox is checked, with the port number '1443' entered. The 'Enable Single Sign On (SSO) for VPN Tunnel' checkbox is also checked. The 'Use external browser as user-agent for saml user authentication' checkbox is unchecked. At the bottom, there are 'Cancel' and 'Save' buttons.

5. Save the configurations.
6. On the main page, click **VPN Name** and select the newly configured VPN from the dropdown list (e.g. "FortiGate SSL VPN"). The **SAML Login** button is displayed.



The screenshot shows the main interface of the FortiClient VPN application. At the top, there is a blue header with the FortiClient logo and the text 'FortiClient VPN'. Below the header, there is a blue bar with the text 'Upgrade to the full version to access additional features and receive technical support.' The main area features a large blue icon of a globe with a laptop and a padlock. Below the icon, there is a 'VPN Name' label and a dropdown menu showing 'FortiGate SSL VPN'. A 'SAML Login' button is displayed below the dropdown menu.

- Click on the **SAML Login** button. The CLP login page is displayed.



- Select the authentication realm configured in the [Create User](#) section and click **Next**.
- Log in using the user configured in the [Create User](#) section. The UAM SAML with FortiGate SSL VPN integration is complete.

Troubleshooting and Debugging for FortiGate CLI

Below are several useful commands for debugging and troubleshooting for the FortiGate CLI:

Error	Possible Solution
Enable debugging for SAML	<pre># diagnose debug application samld -1 # diagnose debug enable</pre>
Enable debugging for SSL VPN	<pre># diagnose debug application sslvpn -1 # diagnose debug enable</pre>

The following are some common errors you may encounter during the FortiGate configuration step and the possible solutions:

Error	Possible Solution
Error: wrong vdom or time expired	<pre># config system global # set remoteauthtimeout 60</pre>

Error	Possible Solution
	# end
Error: 467 No group info in SAML response.	In AccessMatrix, ensure that there are no other Service Providers using the same Id used as issuer value. Refer to the Configure SP in AccessMatrix section for more information.
Error: 471 No user name info in SAML response. Please check SAML configuration.	
i Note: Refer to this article for more information on FortiGate SSL VPN troubleshooting.	

4.2.4. UAM SAML with JIRA

The following sections provide details on the AccessMatrix configurations needed to enable SSO via SAML with a JIRA server. In this deployment, AccessMatrix acts as the SAML Identity Provider (IdP), while the JIRA server acts as the SAML Service Provider (SP).

Configure AccessMatrix as the IdP

i **Note:** All attributes that are unspecified in this section should be left with their default values.

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Identity Provider** and select "DefaultIdentityProvider".
2. Click the **Edit** button.
3. Configure the following basic attributes:

Attribute	Value	Example
*Identity Provider Name	Enter an IdP name.	"DefaultIdentityProvider"
*Id used as issuer:	Enter the domain URL.	"i-sprint.com" or "sim.aaa.i-sprint.io"

4. In the **General** tab, under the **Digital Signature** header, configure the following attributes:

Attribute	Value	Example
Keystore	Enter the keystore path.	"C:\Users\user1\Downloads\saml-idp4.keystore" for Windows, or "/opt/am/conf/saml-idp-keystore.p12" for Linux
Keystore Passphrase	Enter the keystore passphrase.	"passphrase"
Key Alias	Enter the key's alias name.	"Idp" or "sim.aaa.i-sprint.io"

5. In the **Identity Provider System Properties** tab, under the **Service URLs** header, configure the following attributes:

Attribute	Value	Example
Single Sign-On Service URLs	Enter the SSO service URL using the format "<HTTP Binding> - <SSO URL>".	Redirect - https://sim.aaa.i-sprint.io/clp/samlssso" and/or "POST - https://sim.aaa.i-sprint.io/clp/samlssso"

-
6. Save the changes.

Configure the JIRA server as the SP

Create the Service Provider

1. In the Admin Console, navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Identity Provider** and click **Create**.
2. Select the "UAM Identity Provider" implementation and click **Next**.
3. Click the **Edit** button.
4. Configure the following basic attributes:

Attribute	Value	Example
*Service Provider Id	Enter an SP Id.	"samlJIRA"
*Service Provider Name	Enter an SP name.	"samlJIRA"
*Id used as issuer:	Enter the domain URL.	"sp.i-sprint.com" or 'https://ssl.aaa.com:1443/remote/saml/metadata"

5. In the **General** tab, configure the following attributes:

Attribute	Value	Example
Assertion		
*Assertion Consumer Service URLs	Enter the SP's URL for the consumption of the SAML assertion.	"https://ssl.axbsec.com:1443/remote/saml/login"
Name Id		
Value	Select "Id".	-
Attributes		
Attributes to return in the assertion	Enter the same value as the user response Id created in step x.	"samlresponse"

6. Save the changes.
-

Update the AccessMatrix SAML Configurations

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > SAML Configuration**.
2. Ensure that the value of the **Default Identity Provider** attribute to match that of the identity provider being used.

i **Note:** If you have been following the steps in the preceding sections, no changes to this attribute are required, as it will already be set to "DefaultIdentityProvider".

Create the User Response

1. Navigate to **Administration Dashboard > Configuration > Messaging & Response > User Response** and click **Create**, then click **Next**.
2. Configure the following attributes:

Attribute	Value	Example
User Response Id	Enter the SP's URL for the consumption of the SAML assertion.	"https://ssl.axbsec.com:1443/remote/saml/login"
User Response Name	Select "Id".	-
Fields Name (Claim Name)	Enter the same value as the user response Id created in step x.	"samlresponse"
Attribute Name		

3. Add the following **Fields**:

Name (Claim Name)	Attribute Name	Delimiter
"user-name"	"UserName"	Leave this as the default.
"group-name"	"GroupMembership"	Leave this as the default.

4. Add the following **Attributes**:
-

Name (Claim Name)	Attribute Name	Delimiter
"email"	"email"	Leave this as the default.
"email_verified"	"email_verified"	Leave this as the default.



Note: The "email" attribute is required for JIRA Cloud. It may not be required for a JIRA server.

5. Save the changes.

Final Steps

These steps detail how you can set up and test UAM SAML SSO with JIRA using the AccessMatrix Common Login Page (CLP). The full details of these steps are beyond the scope of this guide and will vary depending on your deployment.

1. In the Admin Console, create and configure an authentication realm to be used.
 2. Add the desired user to add in the authentication realm.
 3. Go to `<AM_ROOT>\tomcat\webapps\clp\WEB-INF\classes\am.commonlogin.config.xml` and add the authentication realm into the config file.
 4. Restart the AccessMatrix server.
 5. Test the connection.
-

4.3. User Entitlement and Identity Mapping

About User Entitlement and Identity Mapping

In AccessMatrix UAM, when SAML Service Provider configuration is created, by default, it allows all users to access the service provider. For some deployments, there could be a requirement that certain service providers should be applicable only for certain users. SAML Service Provider configuration in Admin Console allows the granting access to all users be turned off. It will then rely with the administrator to grant access to the user via direct **User - SAML Service Provider** entitlement or alternatively by joining the user to a user group that has been entitled with the access to the SAML Service Provider as described in [Assign User with SAML Application](#).

When a user is single signed-on to a service provider, the user identity of the user in the service provider may be different from the one used in the identity provider. For example, the login user Id used by the user to login to AccessMatrix UAM is bobsmith, but the service provider may require the user's email instead, such as "bobsmith123@example.com". For the SAML SSO to work, AccessMatrix UAM as the SAML Identity Provider needs to provide the user's email instead of the user Id. AccessMatrix UAM allows the NameID value to use the user's email or any other user attribute. This can be done by creating a User Response to specify which user information will be used. The steps are described in [Identity Mapping using User Attribute](#). If there are multiple service providers that require to use the user's email as the identity, they can all be configured to use the same user response or attribute.

In some cases, a certain deployment may want to have a tighter mapping that instead of using a user attribute, it requires an identity to be used solely by the service provider. It may also be that the user identity is in a form of random string that may not be suitable to be viewed as part of user attributes. In this case, AccessMatrix UAM has a secondary account feature. When a secondary account value is set to the user, the system requires it to be assigned to the service provider as well. In other words, the secondary account value is specific to the combination of a user and a service provider. For example, Bob Smith may have a secondary account, "bobsmith123", when accessing the Finance service provider and another secondary account, "ax24-25sfx-gsxx", when accessing the HR service provider. Secondary Account is also useful when multiple user profiles are required for the same user and the same service provider. For example, Jane may have two identities in the Finance service provider, one profile is for accessing Singapore unit, and another profile is for accessing the Hong Kong unit. Refer to [Identity Mapping using Secondary Account](#) for more details.

Assign User with SAML Application

By default, all authenticated users are allowed to access the Service Provider(s). However, to control the user access to the web applications or web services of a Service Provider, there is a need to define which service provider is accessible to a particular user. This can be done by configuring the **SAML Application Privilege** tab of the User or Group's panel.

i **Note:** Prior to user assignment, it is assumed that the Service Provider (business applications) are already configured as described in [SAML Service Provider](#) section. Ensure that the **Allow All Users to Access** attribute is set to "false".

To assign group with SAML application:

1. Navigate to **Operation Dashboard > User & Segment > Group**.
2. Select an existing group or create a new group, if necessary.
3. On the **Create/Edit Group** page, click the **SAML Application Privilege** tab.
4. Click **Add** and select the SAML Service Provider application to assign.
5. Click **OK** and save the changes.

To assign user with SAML application:

1. Navigate to **Operation Dashboard > User & Segment > User**.
2. Select an existing user or create a new user, if necessary.
3. On the **Create/Edit User** page, click the **SAML Application Privilege** tab.

i **Note:** This tab will also display the SAML application(s) inherited from the assigned group membership.

4. Click **Add** and select the SAML Service Provider application to assign.
5. Click **OK** and save the changes.

i **Note:** The assigned SAML application(s) will also be displayed when the user is logged in to the AccessMatrix SSO Portal.

Identity Mapping Using User Attribute

When a user logs in to a business application, the service provider will check the user's credentials with the identity provider (AccessMatrix UAM). The AccessMatrix UAM authenticates the user and then sends

the authorization and authentication data back to the service provider to grant authorization to the user. This authorization data contains user information required by the service provider, and the AccessMatrix system can be configured to return this user-specific information, either by specifying the **Name Id > Value** of the SP to return the user object value (e.g. "Id", "Name", etc.) or by configuring the user response.

To create a user response:

1. Navigate to **Administration Dashboard > Configuration > Messaging & Response > User Response**.
2. Create a user information response collector to map the user identity to the user attribute, e.g. SAMLResp.
3. Then, depending on what the SP needs as user identity, the administrator may need to map the system predefined user's attribute or the user's custom attribute.
 - a. To map the system predefined user's attribute, under **User tab > Fields**, select the predefined user's attribute, e.g. "UserID". Then, specify the value for the **Name (Claim Name)** field, e.g. "Id", which refers to the return name of user response that will be given to the SP.
 - b. To map the user's custom attribute, under **User tab > Attributes**, specify the value for the **Attribute Name** or select from the predefined custom's attribute, e.g. email.



Note: Ensure that the provided custom attribute name is created in the user's page.

Then, specify the value for the **Name (Claim Name)** field, e.g. "IdPEmail", which refers to the return name of user response that will be given to the SP.

4. Save the changes.

Once the user response is created, you can now proceed to configure the [SAML Service Provider in Admin Console](#).

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Service Provider** and click **Create**.
 2. Select "UAM as Identity Provider" for the **Implementation** and click **Next**.
 3. In the **General** tab, under the **Name Id** header, select "User Response" for **Value**.
-

-
4. Then, specify the user response for the **User Response to return as NameId value** attribute. Ensure that only one field/attribute has been specified in the user response configuration. If it contains more than one value, then a random attribute will be chosen.
 5. If the SP requires more attributes as the user identity, select another user response and assign to **Attributes to return in the assertion** attribute. In this case, one or more attributes/fields can be specified in the user response configuration.
 6. Save the changes.

Identity Mapping using Secondary Account

In order to use a secondary account for identity mapping, you need to configure the [SAML Service Provider in Admin Console](#).

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > SAML > Service Provider** and click **Create**.
2. Select "UAM as Identity Provider" for the **Implementation** and click **Next**.
3. In the **General** tab, under the **Name Id** header, select "Secondary Account" for **Value**.
4. (Optional) If you wish to allow users of this SAML application to have multiple profiles, set the value of the **Allow Multiple Profiles** attribute under the **Others** header to "true".
5. Save the changes.

Now you may proceed to the secondary account provisioning for the users of your SAML Service Provider.

Create or Update Secondary Account

There are configure the secondary account of the user:

- Using Admin Console
- Using REST API

Using Admin Console

1. Navigate to **Operation Dashboard > User & Segment > User** and search for the relevant user's name.
 2. Click the user's name. The **View User** page is displayed.
-

3. Click the **Secondary Account** button.
4. Click the **Add Secondary Account** button. The **Add Secondary Account** dialog is displayed.
5. Provide values for each of the fields:
 - **App ID - Type** - Select the relevant SAML Service Provider from the drop-down list (e.g. "sp1 - SAMLServiceProvider").
 - **External Identity** - Enter the external Id (e.g. login Id, unique Id) of the user for the SAML Application (e.g. "demouser@gmail.com").
 - **Profile Name** - Enter a suitable profile name (e.g. "profile1").
6. Click **Apply** to save complete the linking process.
7. Click **Back** to complete the configurations.

Using REST API

- Use authz/2ndacct/update to create or update secondary account.
- Either admin session, dynamic agent session, or an access token can be used. The Edit User admin right is required.



Note: There are various authentication methods for admins/application servers to prove that they have the necessary authorization to use certain AccessMatrix REST APIs:

- Admin session token
- Dynamic agent session
- Access token

Refer to [Using AccessMatrix REST APIs - Configuring Authentication Methods for Accessing AccessMatrix REST APIs](#) for more information on each of these methods.

- Either UserUUID or userId has to be specified.
- profileName is optional when multiple profiles is not enabled.
- Refer to the [AccessMatrix REST API Specification](#) for more details of this API.

Create or Update Secondary Account API Call	
URL: authz/2ndacct/update	
Request	
JSON	
<pre>{ "sessionToken": "10001_Elns3HSnKM3Kju9RuGk8pqp...a-</pre>	

Create or Update Secondary Account API Call

```
7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXXkDsqTg",
  "userId": "demouser",
  "storeId": "",
  "userSegmentId": "",
  "userUUID": "",
  "account": [
    {
      "id": "demouser@gmail.com",
      "appId": "gmail",
      "appObjectClass": "SAMLServiceProvider"
    }
  ]
}
```

Response

JSON

```
{
  "result": 1
}
```

View Secondary Account

There are two ways to view the secondary account of the user:

- Using Admin Console
- Using REST API

Using Admin Console

To view the Secondary Account created for user in Admin Console:

1. Navigate to **Operation Dashboard > User & Segment > User** and search for the relevant user's name.
2. Click the user's name. The **View User** page is displayed.
3. Click the **Secondary Account** button.
4. The secondary account you created for the SAML application will be listed in the **Secondary Account** tab with **Application Id**, **Application Object Class**, **External Identity**, and **Profile Name**.
 - **Application Id** - This will display the SAML Service Provider Id (e.g. sp1).

- **Application Object Class**- This will display the corresponding Object Class of the SAML application - SAMLServiceProvider.
- **External Identity** - This will display the user identity (e.g. login Id, unique Id) of the SAML Application (e.g. demouser@gmail.com).
- **Profile Name** - This will display the profile name configured in the previous section (e.g. profile1)

Using REST API

- Use authz/2ndacct/get to retrieve user's secondary account.
- Either user session, dynamic agent session, or an access token can be used.



Note: There are various authentication methods for admins/application servers to prove that they have the necessary authorization to use certain AccessMatrix REST APIs:

- Admin session token
- Dynamic agent session
- Access token

Refer to [Using AccessMatrix REST APIs - Configuring Authentication Methods for Accessing AccessMatrix REST APIs](#) for more information on each of these methods.

- Either UserUUID or userId has to be specified.
- Multiple SAML Service Providers can be specified under appld.
- Refer to the [AccessMatrix REST API Specification](#) for more details of this API.

Get Secondary Account API Call

URL: authz/2ndacct/get

Request

JSON

```
{
  "sessionToken": "10001_ElNs3HSnKM3Kju9RuGk8pqp...a-7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg",
  "userId": "demouser",
  "storeId": "",
  "userSegmentId": "",
  "userUUID": "",
  "appId": [
    "gmail"
  ],
  "appObjectClass": "SAMLServiceProvider"
}
```

Get Secondary Account API Call

Response

JSON

```
{
  "result": {
    "id": "demouser@gmail.com",
    "appId": "gmail",
    "appObjectClass": "SAMLServiceProvider",
    "profileName": "profile1"
  }
}
```

Delete Secondary Account

There are two ways to delete secondary account:

- Using Admin Console
- Using REST API
 - Delete by App Id
 - (Optionally) Delete by profile

Using Admin Console

1. Navigate to **Operation Dashboard > User & Segment > User** and search for the relevant user's name.
2. Click the user's name. The **View User** page is displayed.
3. Click the **Secondary Account** button.
4. Tick the checkbox beside the secondary account you want to delete.
5. Click the **OK** button.

Using REST API

There are two methods you can use to delete a secondary account via REST API:

1. Delete Secondary Account by App Id

Use `authz/2ndacct/delete` to delete user's secondary account by app Id (SAML Service Provider Id).

- Either admin session, dynamic agent session, or an access token can be used. The Edit User admin right u



Note: There are various authentication methods for admins/application servers to prove that they have the necessary authorization to use certain AccessMatrix REST APIs:

- Admin session token
- Dynamic agent session
- Access token

Refer to [Using AccessMatrix REST APIs - Configuring Authentication Methods for Accessing AccessMatrix REST APIs](#) for more information on each of these methods.

- Either UserUUID or userId has to be specified.
- Multiple SAML Service Providers can be specified under appld
- appObjectClass and appld cannot be empty.
- Refer to the [AccessMatrix REST API Specification](#) for more details of this API.

Delete Secondary Account API Call

URL: authz/2ndacct/delete

Request

JSON

```
{
  "sessionToken": "10001_Elns3HSnKM3Kju9RuGk8pqp...a-7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg",
  "userId": "demouser",
  "storeId": "",
  "userSegmentId": "",
  "userUUID": "",
  "appId": [
    "gmail"
  ],
  "appObjectClass": "SAMLServiceProvider"
}
```

Response

JSON

```
{
  "result": 1
}
```

2. Delete Secondary Account by Profile

- Use authz/2ndacct/deleteByProfile to delete user's secondary account by profile.

- Either user session, dynamic agent session, or an access token can be used.



Note: There are various authentication methods for admins/application servers to prove that they have the necessary authorization to use certain AccessMatrix REST APIs:

- Admin session token
- Dynamic agent session
- Access token

Refer to [Using AccessMatrix REST APIs - Configuring Authentication Methods for Accessing AccessMatrix REST APIs](#) for more information on each of these methods.

- appObjectClass and appId cannot be empty.
- Refer to the [AccessMatrix REST API Specification](#) for more details of this API.

Delete Secondary Account API Call

URL: authz/2ndacct/deleteByProfile

Request

JSON

```
{
  "sessionToken": "10001_ELnS3HSnKM3Kju9RuGk8pqp...a-7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg",
  "userId": "demouser",
  "storeId": "",
  "userSegmentId": "",
  "userUUID": "",
  "account": [
    {
      "id": "",
      "appId": "gmail",
      "appObjectClass": "SAMLServiceProvider",
      "profileName": ""
    }
  ]
}
```

Response

JSON

```
{
  "result": 1
}
```

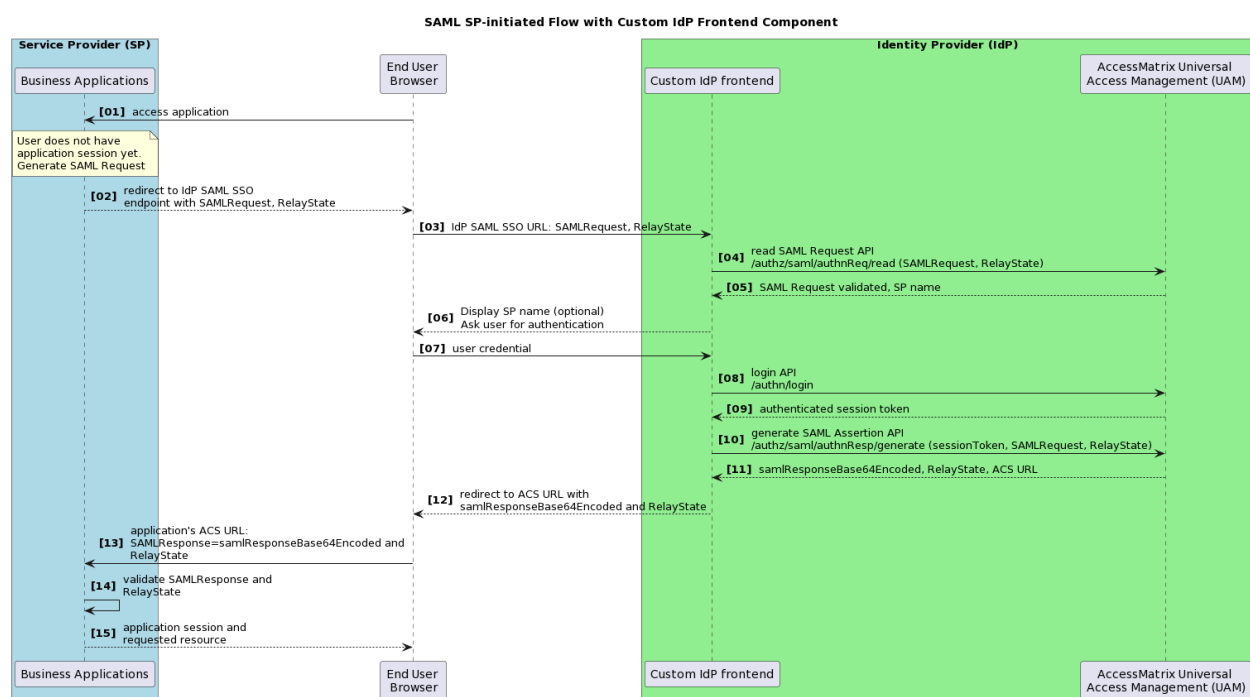
4.4. Custom Identity Provider Front End

AccessMatrix UAM allows customisation of the IdP front end based on the following use cases:

- Service Provider-Initiated
- Identity Provider-Initiated

Refer to [SP-initiated vs IdP-initiated](#) section for more details.

Service Provider-Initiated Sequence Diagram

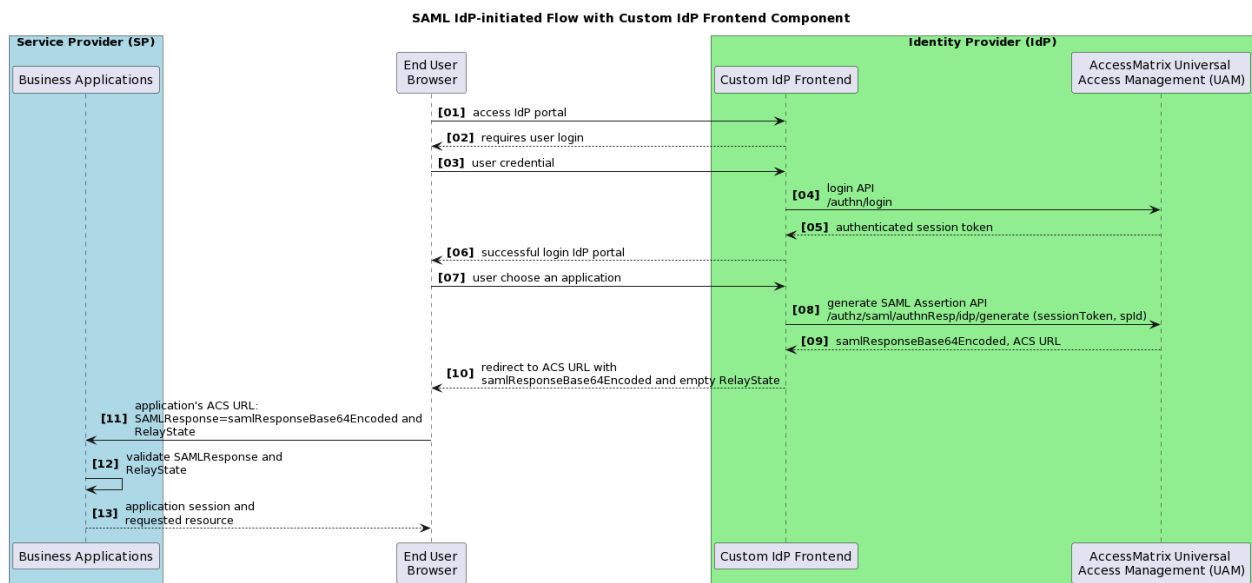


In the sequence diagram above:

1. The user access the business applications.
 2. If the user is not authenticated yet, the application will generate a SAML request and redirect this request to the custom IdP together with the relay state.
 3. The custom IdP needs to create SAML SSO endpoint to receive the SAMLRequest and RelayState parameters.
 4. The custom IdP calls the `/authz/saml/authnReq/read` API to validate the SAML request.
 5. After successful validation of the SAML request, the SP name will be returned and can optionally be displayed on the login page.
-

6. The user then provides the credential on the login page and the custom IdP calls the /authn/login API to authenticate the user.
7. Once the user has been authenticated, the session token will be returned to the custom IdP and set this session token in the browser cookie.
8. The custom IdP then calls the /authz/saml/authnResp/generate API to generate the SAML assertion. This API returns a Base64-encoded SAML response, relay state and ACS URL.
9. The custom IdP needs to create a redirection page that has a form with hidden inputs, SAMLResponse and RelayState, to send the Base64-encoded SAML response and relay state using POST method to the SP's ACS URL. The page should automatically submit the form (or with some delay if required).
10. The application decodes the Base64-encoded SAML response and validates both the SAML response and relay state.
11. After successful validation, the application returns the application session to the browser and allows the user to access the application and its requested resources.

Identity Provider-Initiated Sequence Diagram



In the sequence diagram above:

1. The user access the custom IdP portal (login page) and requires the user to log in.
2. The user provides the credential on the login page.
3. The custom IdP calls the /authn/login API to authenticate the user.

-
4. Once the user has been authenticated, the session token will be returned to the custom IdP and set this session token in the browser cookie.
 5. Once the user is successfully logged in to the custom IdP portal, the user then chooses an application inside the custom IdP portal.
 6. The custom IdP then calls the `/authz/saml/authnResp/idp/generate` API to generate the SAML assertion. This API returns a Base64-encoded SAML response and ACS URL.
 7. The custom IdP needs to create a redirection page that has a form with hidden inputs, `SAMLResponse` and `RelayState`, to send the Base64-encoded SAML response and empty relay state using POST method to the SP's ACS URL. The page should automatically submit the form (or with some delay if required).
 8. The application decodes the Base64-encoded SAML response and validates both the SAML response and relay state.
 9. After successful validation, the application returns the application session to the browser and allows the user to access the application and its requested resources.

Login

- Use `authn/login` to authenticate the user. In order to generate SAML Response, it requires a valid user session (`sessionToken`). This login steps is to serve that purpose: to identify the user and get the user session.
- The `userId`, `password`, and `realmId` must be specified.
- Refer to the [AccessMatrix REST API Specification](#) for more details of this API.

Login API Call	
URL: <code>authn/login</code>	
Request	
JSON	<pre>{ "userId": "demouser", "password": "p@sw0rd1", "realmId": "40151" }</pre>
Response	
JSON	

Login API Call
<pre> { "result": { "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp...a-7HCiSnx0xhlGuIjd8YO8bg3HLWHp6yxvDLH6L0OKXKkDsqtg", ... } } </pre>

Additional Authentication Options

- The example code above is for a simple password-based authentication. AccessMatrix system supports other types of authentication, especially when the AccessMatrix UAS license is enabled. You may refer to the following documents:
 - End-to-end Encryption Authentication:** [UAS Pseudo E2EEA Implementation](#), [UAS E2EEA with SafeNet ProtectServer HSM Implementation](#), [UAS E2EEA with Thales payShield 10K HSM Implementation](#), and other similar documents.
 - Token-based Authentication:** [UAS OneSpan Token Implementation Guide](#), [UAS SMS OTP Implementation Guide](#), [UAS OneSpan FM with SafeNet ProtectServer HSM Implementation Guide](#), and other similar documents.
- It is also possible to externalize the user authentication (e.g. Social network login, SSO via OIDC, Singapore SingPass Mobile)
 - [UAM OIDC Relying Party Implementation Guide](#), [UAM Social Network Login Config and API Integration Guide](#), and other similar documents.

Read Authentication Request

- Use authz/saml/authnReq/read to read SAML authentication request.
- The authenticationRequest and relayState must be specified.
- Refer to the [AccessMatrix REST API Specification](#) for more details of this API.

Read Authentication Request API Call
URL: authz/saml/authnReq/read
Request
JSON

Read Authentication Request API Call	
<pre>{ "authenticationRequest": "bM441nuRIzAjKeMM8RhegMFjZ4L4xPBn5IocWuzu.../QvFvuB6BcJaXaz0h7NwGhHROUTe6MoGJKMPE=", "relayState": "https://sp.abcbank.com/relaystate" }</pre>	
Response	
JSON	
<pre>{ "result": { "spName": "abcbank", ... } }</pre>	

Generate Authentication Response

SP-initiated

- Use authz/saml/authnResp/generate to generate authentication response.
- The sessionToken, authenticationRequest, and relayState must be specified.
- Refer to the [AccessMatrix REST API Specification](#) for more details of this API.

Generate Authentication Response API Call (SP-initiated)	
URL: authz/saml/authnResp/generate	
Request	
JSON	
<pre>{ "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp...a-7HCiSnx0xhlGuIjd8YO8bg3HLWHP6yxvDLH6L0OKXKkDsqtg", "authenticationRequest": "bM441nuRIzAjKeMM8RhegMFjZ4L4xPBn5IocWuzu.../QvFvuB6BcJaXaz0h7NwGhHROUTe6MoGJKMPE=", "relayState": "https://sp.abcbank.com/relaystate"}</pre>	
Response	
JSON	

Generate Authentication Response API Call (SP-initiated)

```
{
  "result": {
    "samlResponseBase64Encoded":
"BDTset1RTyuj3RYJ8sERyhUBswXt5Tcat...0GcerdQZFR3CW4CdzIvsQCM",
    "relayState": "https://sp.abcbank.com/relaystate",
    "acsUrl": "https://sp.abcbank.com/saml/acs",
    ...
  }
}
```

IdP-initiated

- Use authz/saml/authnResp/idp/generate to generate authentication response.
- The sessionToken, relayState, and spId must be specified.
- Refer to the [AccessMatrix REST API Specification](#) for more details of this API.

Generate Authentication Response API Call (IdP-initiated)

URL: authz/saml/authnResp/generate

Request

JSON

```
{
  "sessionToken": "10001_ELns3HSnKM3Kju9RuGk8pqp...a-
7HCiSnx0xhlGuIjd8YO8bg3HLWHp6yxvDLH6L0OKXKkDsqtg",
  "relayState": "",
  "spId": "abcBank"
}
```

Response

JSON

```
{
  "result": {
    "samlResponseBase64Encoded":
"BDTset1RTyuj3RYJ8sERyhUBswXt5Tcat...0GcerdQZFR3CW4CdzIvsQCM",
    "relayState": "",
    "acsUrl": "https://sp.abcbank.com/saml/acs",
    ...
  }
}
```

Sample SAML IdP Front End

The AccessMatrix installation package comes with a SAML IdP front end (REST) WAR file (e.g. **AccessMatrix-x.x.x.xxxx-samlidp-rest-sample.war**). This sample web application illustrates the use of the APIs mentioned above for building a custom IdP front end. This application **must not** be used in a production environment.

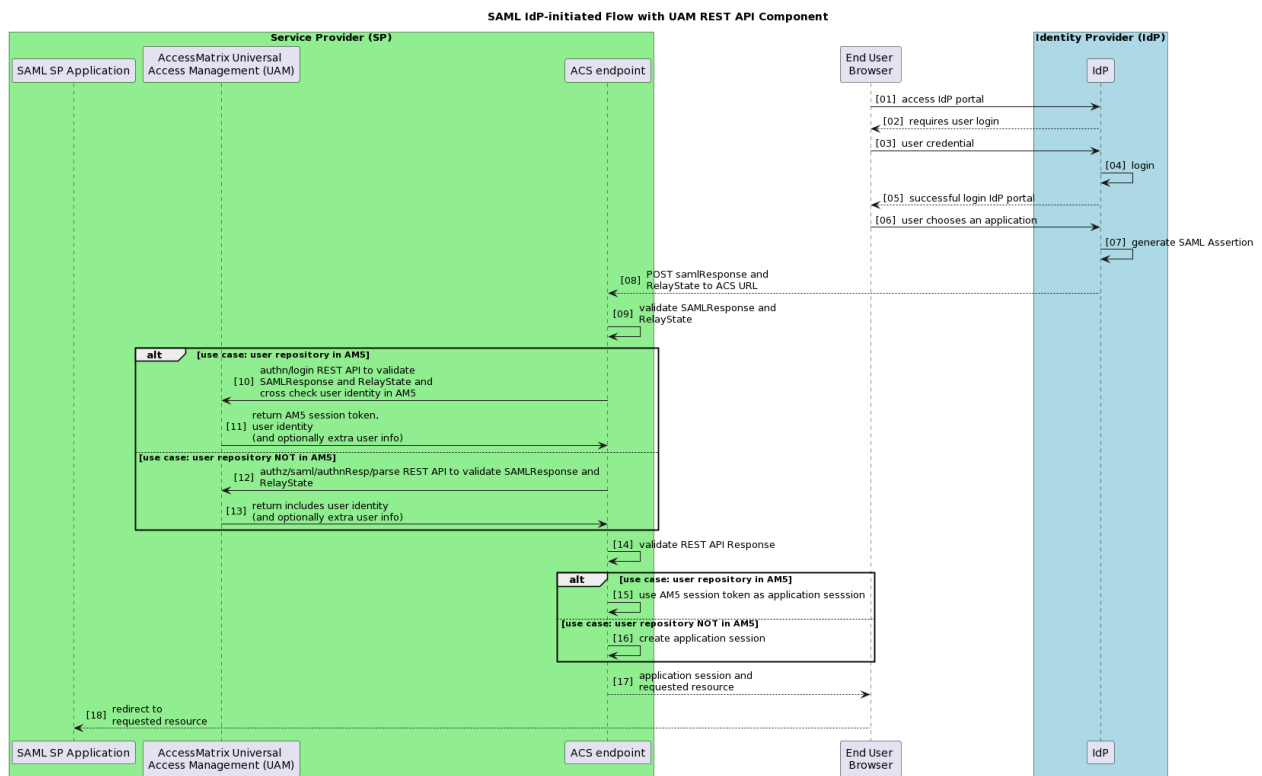
5. UAM SAML Service Provider ACS Deployment

This section provides the configuration and deployment details of the SAML SP Assertion Consumer Service (ACS) web application with AccessMatrix Universal Access Management (UAM).

About UAM SAML SP ACS

The UAM SAML SP ACS is a web application that consumes SAML assertions and parses a signed SAML responses from the Identity Provider (IdP) to complete SAML single sign-on (SSO). After the process is done, it will set AccessMatrix session token cookie to allow cookie-based web SSO to use the user session.

Sequence Diagram



The flow is described below:

1. The target application requires user to SSO using SAML; thus, it redirects the user browser to the SAML SP ACS Request endpoint.
2. The SAML SP ACS generates the SAML Request and redirects the user browser to the SAML Identity Provider (IdP) with the SAML Request.

-
3. The user is authenticated by the IdP (authentication method or flow depends on the IdP).
 4. The IdP sends a response to the SAML SP ACS endpoint, where the SAML SP ACS processes it (via AccessMatrix authentication).
 5. Once it is successful, the SAML SP ACS sets the AccessMatrix session (WSASID cookie) and then redirects the user to the target application based on the target URL configured.

5.1. Deployment

i **Note:** The term `<ACMATRIX_ROOT>` is used to refer to the root or installation directory of AccessMatrix on both Windows and Unix platforms.

Pre-installation Checklist

Ensure that the following servers are configured prior to deployment of UAM SAML SP ACS web application:

- The AccessMatrix server should have been deployed and successfully started.
- The Identity Provider server should have been configured and should be running.

Deployment Steps

1. Secure the SAML SP ACS WAR file from the AccessMatrix CD-Image. The filename format is **AccessMatrix-x.x.xxxx-GA-samlsp-acs.war**, e.g. **AccessMatrix-5.6.3.6308-GA-samlsp-acs.war**.
2. (Optional) For easier reference, rename the WAR file to **samlsp-acs.war**.
3. SAML SP ACS is a stateless web service, therefore you can deploy the WAR file (**samlsp-acs.war**) to any web application container supported by AccessMatrix core products, e.g. Apache Tomcat.
4. If using Tomcat, create a *samlsp-acs* folder under `<ACMATRIX_ROOT>\tomcat\webapps\`.
5. Extract the content of **samlsp-acs.war** to this folder.

SAML SP ACS Components

Below is the list of the folders and sub-folders inside the SAML SP ACS WAR file:

Folder Name	Sub-folder/File Name	Description
web	css\style.css	Styling CSS file.
	images	Contains the icons and other images.
	WEB-INF\classes\log4j.properties	Contains the configurations of log level and log output file location.
	WEB-INF\lib	The libraries used for SAML SP ACS.

Folder Name	Sub-folder/File Name	Description
	WEB-INF\web.xml	Contains the configuration of REST connection to AccessMatrix server, cookies setting, target URL after authenticated user is granted SP service, servlet mapping URL pattern to include login module Id for "SAML SP Request Servlet" & realm Id for "SAML SP ACS Servlet".
	error.jsp	The page to display an error message.
	spservice.jsp	Renders a welcome page after SAML response has been validated where authenticated user is granted an SP service.
 Note: Only retain the <i>WEB-INF</i> folder and error.jsp when deploying into the production environment.		

5.2. Configuration

Setup Steps

This section defines the steps relating to the configuration of the UAM SAML SP ACS web application.

- **Set up Service Provider (SP) ACS Server**

In this deployment, AccessMatrix server acts as the service provider. Refer to [Set up SP ACS Server](#) for configuration details.

- **Configure SAML SP ACS Configuration File**

Before the UAM SAMP SP ACS web application can be used with AccessMatrix server, the SAML SP ACS configuration file must be properly configured. Refer to [Configure SAML SP ACS Configuration File](#) for configuration details.

- **Start the SP and IdP Servers**

Start up the IdP server and restart the SP server (i.e. AccessMatrix server) after performing the configurations mentioned above.

- **Test the SAML SSO**

Open a browser and visit the SAML SP ACS Request endpoint as configured in the SAMLSPRequestServlet URL pattern of the [Configure SAML SP ACS Configuration File](#) step.

Alternatively, the target application that requires SSO can provide the user with a link to redirect the user to the SAML SP ACS Request endpoint to automatically initiate the SAML SSO process.

- **Troubleshoot REST API Errors**

If necessary, troubleshoot any errors that may be encountered. Refer to [Troubleshoot Errors](#) for a list of errors that might be encountered and their possible causes.

Set Up SP ACS Server



Note: Prior to configuration, ensure that AccessMatrix server is up and running.

1. Login to AccessMatrix server.
 2. Configure the default or create a new SAML Identity Provider (IdP). Refer to [UAM Administration Guide - Identity Provider \(UAM as Service Provider\) Page Attribute Description](#) section for the attribute details.
 3. Create a SAML Service Provider (SP). Refer to [UAM Administration Guide- Service Provider \(UAM as Service Provider\) Page Attribute Description](#) section for the attribute details.
-

-
4. Navigate to **Administration Dashboard > Configuration > Authentication > User Lookup Module** and create an **SAML Web Browser SSO Lookup and Login** user lookup module.
 5. Set **Is IDP initiated Authentication Response** attribute to "No".
 6. Configure the rest of the **Configuration** tab attributes. Refer to [Common Administration Guide - SAML Web Browser SSO Lookup and Login](#) section for the attribute details.
 7. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module** and create a **SAML Web Browser SSO Lookup and Login** login module. Refer to [Common Administration Guide - SAML Web Browser SSO Lookup and Login](#) section for the attribute details.
 8. Create a new authentication realm and specify the lookup and login modules created in previous steps. Save the changes.
 9. Assign the authentication realm to the user or enable auto assignment in the authentication realm.

Configure SAML SP ACS Configuration File

To configure SAML SP ACS configuration file, navigate to `<ACMATRIX_ROOT>\tomcat\webapps\samlsp-ac\web\WEB-INF` and modify the **web.xml** file. The content is shown below:

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://java.sun.com/xml/ns/javaee"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
  id="WebApp_ID" version="3.0">

  <display-name>SAMLSP ACS</display-name>

  <context-param>
    <param-name>restAPIURL</param-name>
    <param-value>http://localhost:8080/am5/rest</param-
value>
  </context-param>

  <context-param>
    <!--cookie domain for requestId & WSASID cookies-->
    <param-name>cookieDomain</param-name>
    <param-value>sp.i-sprint.demo</param-value>
  </context-param>
  <context-param>
    <!--cookie age (in seconds) for requestId & WSASID
cookies-->
    <param-name>cookieAge</param-name>
```

```

        <param-value>120</param-value>
    </context-param>

    <servlet>
        <servlet-name>SAMLSPRequestServlet</servlet-name>
        <servlet-
class>com.isprint.am.saml.sp.acs.SAMLSPRequestServlet</servlet-
class>
        <init-param>
            <!-- The target URL (full URL path) or SP relay state-
->
                <param-name>targetURL</param-name>
                <param-value>http://sp.i-sprint.demo/samlsp-
acs/spservice.jsp</param-value>
            </init-param>
        </servlet>
    <servlet>
        <servlet-name>SAMLSPACSServlet</servlet-name>
        <servlet-
class>com.isprint.am.saml.sp.acs.SAMLSPACSServlet</servlet-
class>
    </servlet>

    <servlet-mapping>
        <!--Servlet path "/req" follow by "<loginModuleId>".
Configure the loginModuleId with
SAMLWebBrwsSSOLookupLoginModule handler -->
        <servlet-name>SAMLSPRequestServlet</servlet-name>
        <url-pattern>/req/saml1gn</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <!--Servlet path "/acs" follow by "<realmId>". Configure
the realmId that contain the user lookup module and login
module with SAMLWebBrwsSSOLookupLoginModule handler -->
        <servlet-name>SAMLSPACSServlet</servlet-name>
        <url-pattern>/acs/samlrlm</url-pattern>

    </servlet-mapping>
</web-app>

```

The table below explains the important properties to configure in the **web.xml** file:

Property Name	Description	Remarks
restAPIURL	This property refers to the AccessMatrix server REST API Service URL. Example: <i>http://localhost:8080/am5/rest</i>	This establishes the connection between SAMP SP ACS web application and AccessMatrix server.
cookieDomain	Specify the cookie domain for requestId & WSASID cookies.	-

Property Name	Description	Remarks
cookieAge	Specify the cookie age of requestId and WSASID cookies, in seconds.	Default: 120
targetURL	This is the target application page after the SAML response has been validated where authenticated user is granted with SP service.	It must be in a full URL path.
SAMLSPRequestServlet url-pattern	Refers to the request servlet path. Parameter value should be: /req/<loginModuleId>.	<loginModuleId> is referring to the "SAML Web Browser SSO Lookup and Login" login module Id.
SAMLSPACSServlet url-pattern	Refers to the SP ACS servlet path. Parameter value should be: /acs/<realmId>.	<realmId> is referring to the authentication realm that contains the "SAML Web Browser SSO Lookup and Login" user lookup module and login module.

Save the changes.

Configure Connection Pool Settings

Connection pooling refers to the method of creating a pool of connections and caching those connections so that it can be reused again. To configure the connection pool parameters:

1. Navigate to <ACMATRIX_ROOT>\tomcat\webapps\samlsp-acs\web\WEB-INF and open **web.xml**.
2. Look for the Connection Pool Configuration keywords and modify the parameter values, if necessary.

XML

```
<!-- Connection Pool Configuration -->
<context-param>
  <param-name>am.hc4.timeout.conn</param-name>
  <param-value>20000</param-value>
</context-param>
<context-param>
  <param-name>am.hc4.pool.maxconn_per_route</param-
name>
```



```
<param-value>50</param-value>
</context-param>
```

3. You may add more connection pool parameters by referring to the following section.
4. Save the changes.
5. Restart the AM5 server for the new changes to take effect.

List of Settable Parameters

For more details on the behavior of the configuration, refer to HTTP Client 4.5 documentation.

Parameter Name	Description
Connection Pool	
am.hc4.pool.maxconn_per_route	The total number of connections limit to a single route (host and port). Default: 50
am.hc4.pool.maxconn_total	The total maximum connections available in the connection pool. Default: 100
am.hc4.pool.ttl_ms	Defines the maximum life span of persistent connections regardless of the allowed time-to-live returned by server. Default: 180 sec
Retry	
am.hc4.retry.times	The maximum number of times a method will be retried. Default: 1
am.hc4.retry.request_sent_retry	Set to "true" to retry methods that have successfully sent their requests. Default: false
Timeout	
am.hc4.timeout.conn	Determines the timeout in seconds until a connection is established. Default: 10 sec
am.hc4.timeout.socket	Determines the timeout when waiting for receiving or sending data (socket read/write timeout). Default: 30 sec

Parameter Name	Description
am.hc4.timeout.conn_request	Maximum wait time to get a connection (waiting for connection pool to return a connection). Default: 30 sec
SSL	
am.hc4.ssl.ciphers-suites	Specifies the ciphers that facilitate secure sockets layer/transport layer (SSL/TLS).
am.hc4.ssl.protocols	The networking protocol used to secure connections between web clients and web servers over an insecure network.
am.hc4.ssl.keystore	The Java keystore file that contains the key used for client-authentication SSL.
am.hc4.ssl.keystore-passphrase	The password of the Java keystore.
Proxy	
am.hc4.proxy.host	The host name or IP address of a proxy server that you want to connect to.
am.hc4.proxy.port	The number of the port that the proxy server uses to listen for client connections.
am.hc4.proxy.username	The user name that you use to access the proxy server.
am.hc4.proxy.password	The password that you use to access the proxy server.
am.hc4.proxy.ntlm	Indicate whether NTLM proxy authentication is enabled. Default: false
am.hc4.proxy.ntlm.domain	The domain where credentials are intended to authenticate with.
am.hc4.proxy.ntlm.host	The host or workstation name of the computer originating the authentication request.
Socket	
Advanced Java Socket object configurations. In most cases, there is no need to configure this. Refer to Java SocketOptions for more details, including the default values. Some settings may not work on certain Operating Systems (OS) and require setting them at the OS-level instead.	
am.hc4.socket.receive-buffer-size	Sets the size of the receive buffer for newly created sockets.
am.hc4.socket.send-buffer-size	Sets the size of the send buffer for newly created sockets.

Parameter Name	Description
am.hc4.socket.keep-alive	Indicate whether or not to have socket keep alive turned on.
am.hc4.socket.linger	Sets the information about the linger state of the associated socket.
am.hc4.socket.timeout	Defines the socket timeout in seconds, which is the time waiting for data - after establishing the connection. It is the maximum period of inactivity between two consecutive data packets. This setting is the same as am.hc4.timeout.socket.
am.hc4.socket.tcp-no-delay	Sets the socket TCP_NODELAY flag.

Troubleshoot REST API Errors

After setting up the UAM SP ACS web application, you may encounter errors for a variety of reasons. To aid in the troubleshooting process, you can refer to the following list of REST API errors for each SAML SP servlet and their possible causes:

i Note: Words enclosed in "<>" represent user-defined values.		
Status Code	Error Message	Possible Causes
REST API Errors in SAMLSPRequestServlet (/authn/preauthenticate REST API)		
16000: SAMLException	-	Failed to generate signature for the redirect binding algorithm Id <AlgId>.
		Failed to sign the SAML request.
16001: InvalidRequest	-	The Service Provider <issuer> was not recognized.
		The Identity Provider Id <IDPId> was not recognized.
16006: ConfigurationException	-	The digital signature was not properly configured.
		The signing key algorithm value was not available.

Status Code	Error Message	Possible Causes
		Could not map to the algorithm URI.
REST API Errors in SAMLSPACSServlet (/authn/login REST API)		
10001: UserNotFound	Failed to lookup user by user identity	Failed to look up user based on the resolved user identity, as the user was not found based on the specified <userIdMapping>
	Failed to lookup user by using user <userIdMapping>	Failed to look up user due to a runtime error.
	Failed to lookup user by SAML Authn response	One of the following errors related to invalid requests were encountered: - The Service Provider <issuer> was not recognized. - The Identity Provider Id <IDPid> was not recognized.
		One of the following errors related to assertion decryption were encountered: Could not decrypt the SAML assertion.
		One of the following errors related to invalid responses were encountered: - The audience restriction was not matched. - The response was reused. - The assertion was reused. - Could not find the issuer in the response or assertion. - Failed to retrieve the user identity.
		One of the following errors related to invalid time were encountered:

Status Code	Error Message	Possible Causes
		- The assertion issue time was outside the acceptable range (SP's assertion time window). - The session expired, based on the session expiry time specified by the IdP.
		One of the following errors related to signature validation were encountered: - Failed to validate the SAML object signature as the Signature element could not be found. - The detected signature algorithm was not an approved signature algorithm. - Failed to verify the digest of each reference in the signature.
		One of the following errors related to validation were encountered: There was a mismatch of the response's authentication context.
10009: PasswordNotFound	AuthnResponse string cannot be empty or null	The AuthnResponse string was left empty or had a null value.

6. UAM SAML Reports

Auditing for User/ User Group SAML SP Privileges

The AccessMatrix UAM authorization server audits the granting/removal of SAML Service Provider application privileges to/from users and user groups. The administrator can create a report to view the audited events, which are as follows:

Event Type	Description	Fields
Principal- AddSAMLSPPrivilege	This audit event is raised when a user/user group is granted a SAML Service Provider application privilege.	SPId
Principal-RemoveSAMLSPPrivilege	This audit event is raised when a user's/user group's SAML Service Provider application privilege is removed.	SPId

To view the above audit events, generate either an Events Report or User Activities Report in the Admin Console by following the steps below:

1. Navigate to either:
 - a. Events Report - **Report Dashboard > System & Admin > Historical > Events.**
 - b. User Activities Report - **Report Dashboard > Segment & User > Historical > User Activities.**
2. For **Event Category**, ensure **All Event Categories** is selected.
 - a. Alternatively, click **Select Event Category** and select **Principal**.
3. Input the report criteria, select the report format (i.e. "HTML", "PDF" or "CSV") and click the **Generate** button.

7. Appendices

A. FAQ

Create a CER file from SAML IdP Metadata XML File

1. Open and edit the metadata file in a text editor of your choice, e.g. Notepad++.
2. Copy the certificate data after <md:KeyDescriptor use="signing"><ds:KeyInfo

xmlns:ds="http://www.w3.org/2000/09/xmldsig#"><ds:X509Data><ds:X509Certificate>, e.g.:

```
MIIDYDCCAkigAwIBAgIJALLJPAyvf2sjMA0GCSqGSIb3DQEBBQUAMCkxJzAIBgNV
BAMTHkxpdmUgSUQgU1RTIFNpZ25pbmcgUHVibGljIEtleTAeFw0xNDA3MTgxOTUz
NDBaFw0xOTA3MTcxOTUzNDBaMCkxJzAIBgNVBAMTHkxpdmUgSUQgU1RTIFNpZ25p
bmcgUHVibGljIEtleTCCASlwdQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBANYD
KgByFZdqTnnpF4IfIp4i2XLg2rLlo+mu4DmW9gRLIBJCNc7YESUxpKzuFYaAnd8
fWsDigJZTXbhOQApSpw4xXFnor2vJ1zm94LtqjcVEXTjUml5gAIS4pwuOU3ZF0/0
eTG0gDYp4a0L/mzzTRsnwe/8WMPiE75Bq2zAyAZ9aePvI3QX7cXYLPfeK4QTgK3B
5lwe1wWu3y5oQidjcSok8Fr80xzuCYuOa+ZUK3JibpLLCrT4uwiqf+KREDSdc4b
PPIq0PWI4sQr1tha8yypRSvOH+/MxcfSRSnl6Uc+gm8nVEEWWlu4hhu6NIfG91mM
UqJuzkgLCi6Gov6JS8UCAwEAAaOBijCBhzAdBgNVHQ4EFgQUUnQoq7sl3R8rde4sQ
s6nGEbJm3LcwWQYDVR0jBFIwUIAUnQoq7sl3R8rde4sQs6nGEbJm3LehLaQrMCkx
JzAIBgNVBAMTHkxpdmUgSUQgU1RTIFNpZ25pbmcgUHVibGljIEtleYIJALLJPAyvf
2sjMAsGA1UdDwQEAwIBxjANBgkqhkiG9w0BAQUFAAOCAQEAF4jaNhKzRG3k+52W
oM9nnlSP7rIWleWwH6EQGUIF6ozSP/03gYMAAdqpdhww5zNwKzi7TQVbDC0pgq/tq
zHv6JEI0R4B6h7/TJ1pYPxdvIFQrE27RHESltH/m+5UkVnayLqRD3/fi4zf4aEpx
SDZ73MCR5LanPGqvlAMz29AL3g1ynj+eu7xMfFsM/8+qJaCXuxT5/30eeLEe+PYi
kA/PhEwp+qkDQWPvdAwEghuUaFvtKAqDZierjpGzHZnYkXTTDTHVe1iP7tsAJH5q
K3qdcv3UGPyZrjC/letJcAcnwVoZQ93v2ieGfcKKN+PFN9M59/BkPo62HPoGNNx
2ZDQaQ==
```

3. Create a new file and paste the copied certificate data. A valid .cer file contains a BEGIN and END certificate declaration. Your file should look something like this:

```
-----BEGIN CERTIFICATE-----
MIICQzCCAaygAwIBAgIETkx4kTANBgkqhkiG9w0BAQUFAADBmMQswCQYDVQQGEwJTRzELM
AkGA1UE
...
mKrcb2IZl6ltTX78Q==
-----END CERTIFICATE-----
```

4. Save the file using the .cer file extension, e.g. <filename>.cer.
 5. You should be able to view its content as a valid certificate now.
-

B. Authentication Context Classes

Authentication Context Class	Description
urn:oasis:names:tc:SAML:2.0:ac:classes:AuthenticatedTelephony	Indicates authentication using a line number, a user suffix, and a password.
urn:oasis:names:tc:SAML:2.0:ac:classes:InternetProtocol	Indicates authentication using a provided IP address.
urn:oasis:names:tc:SAML:2.0:ac:classes:InternetProtocolPassword	Indicates authentication using a provided IP address, in addition to a username/password.
urn:oasis:names:tc:SAML:2.0:ac:classes:Kerberos	Indicates authentication using a password to a local authentication authority to acquire a Kerberos ticket, which is then used for subsequent network authentication.
urn:oasis:names:tc:SAML:2.0:ac:classes:MobileOneFactorContract	Indicates mobile contract customer registration procedures and authentication using single-factor authentication.
urn:oasis:names:tc:SAML:2.0:ac:classes:MobileOneFactorUnregistered	Indicates authentication with a mobile device.
urn:oasis:names:tc:SAML:2.0:ac:classes:MobileTwoFactorContract	Indicates mobile contract customer registration procedures and authentication using two-factor authentication.
urn:oasis:names:tc:SAML:2.0:ac:classes:MobileTwoFactorUnregistered	Indicates authentication using two-factor authentication, such as with a mobile device and a password.
urn:oasis:names:tc:SAML:2.0:ac:classes:NomadTelephony	Indicates "roaming" (for example, using a phone card) and authentication

Authentication Context Class	Description
	using a line number, a user suffix, and a password.
urn:oasis:names:tc:SAML:2.0:ac:classes:Password	Indicates authentication to an authentication authority through the presentation of a password over an unprotected HTTP session.
urn:oasis:names:tc:SAML:2.0:ac:classes:PersonalTelephony	Indicates authentication via the provision of a fixed-line telephone number and a user suffix, transported via a telephony protocol such as Asymmetric Digital Subscriber Line (ADSL).
urn:oasis:names:tc:SAML:2.0:ac:classes:PGP	Indicates authentication using a digital signature, where the key was validated as part of a PGP Public Key Infrastructure.
urn:oasis:names:tc:SAML:2.0:ac:classes:PasswordProtectedTransport	Indicates authentication to an authentication authority through the presentation of a password over a protected session.
urn:oasis:names:tc:SAML:2.0:ac:classes:PreviousSession	Indicates successful authentication to an authentication authority using any supported authentication context in the past.
urn:oasis:names:tc:SAML:2.0:ac:classes:Smartcard	Indicates authentication to an authentication authority using a smartcard.
urn:oasis:names:tc:SAML:2.0:ac:classes:SmartcardPKI	Indicates authentication to an authentication authority using a two-factor authentication mechanism that includes the use of a smartcard with an enclosed private key and a PIN.

Authentication Context Class	Description
urn:oasis:names:tc:SAML:2.0:ac:classes:SoftwarePKI	Indicates authentication using an X.509 certificate stored in software to authenticate to the authentication authority.
urn:oasis:names:tc:SAML:2.0:ac:classes:SPKI	Indicates authentication using a digital signature, where the key was validated via an SPKI Infrastructure.
urn:oasis:names:tc:SAML:2.0:ac:classes:SecureRemotePassword	Indicates authentication using a Secure Remote Password (SRP), the cryptographically strong network authentication mechanism specified in the Internet Standards Working Group Request For Comments 2945 (RFC2945).
urn:oasis:names:tc:SAML:2.0:ac:classes:Telephony	Indicates authentication via the provision of a fixed-line telephone number, transported via a telephony protocol such as Asymmetric Digital Subscriber Line (ADSL).
urn:oasis:names:tc:SAML:2.0:ac:classes:TimeSyncToken	Indicates authentication using a time synchronization token.
urn:oasis:names:tc:SAML:2.0:ac:classes:TLSClient	Indicates authentication by means of a client certificate, secured with the SSL/TLS transport.
urn:oasis:names:tc:SAML:2.0:ac:classes:unspecified	Indicates authentication using unspecified means.
urn:oasis:names:tc:SAML:2.0:ac:classes:X509	Indicates authentication using a digital signature, where the key was validated as part of an X.509 Public Key Infrastructure.

Authentication Context Class	Description
urn:oasis:names:tc:SAML:2.0:ac:classes:XMLDSig	Indicates authentication using a digital signature according to the processing rules specified in the XML Digital Signature specification [XMLSig].

8. Quick Setup Cheatsheet

Purpose of the cheatsheet: A quick reference on what to configure in AM console to achieve a certain use-case objective. It is meant to show 'how-to' of typical usage.

The topic will be in sections by use cases.

Set up Basic SAML SSO where AM acts as SAML IdP backend to authenticate and provide user identity info

1. Create a new Identity Provider (UAM as Identity Provider) or edit DefaultIdentityProvider
2. Navigate to Configuration > Unified SSO > Federated SSO > SAML > Identity Provider
3. Fill in the information for **Keystore, Keystore Passphrase, Key Alias**, which is the keypair used for signing
4. Fill in the information for **Single Sign-On Service URLs**, which is the URL for the SP web application to send the SAML request to for SSO. eg. `http://[host]/clp/samlssso`
5. Create a new Service Provider (UAM as Identity Provider) for the SP web application using this SAML SSO, using the info provided by SP
6. Navigate to Configuration > Unified SSO > Federated SSO > SAML > Service Provider
7. Fill in the information for **Id used as issuer**, an identifier of the SP. eg. `sp.i-sprint.com`
8. Fill in the information for **Assertion Consumer Service URLs**, which is the URL to which the SP web application wants the SAML response to be sent. eg. `http://[host]/samlsp-sample-app/acs`
9. Fill in the information for **Name Id Value**, the user identity provided to SP for SSO in SAML Response. eg. `Id`

AM is now ready to provide SAML SSO service as IdP backend via APIs (ie `authz/saml/authnReq/read`, `authz/saml/authnResp/generate`, `authz/saml/authnResp/idp/generate`, `authz/saml/authnResp/preAssessGenerate`).

Refer to [SAML Web SSO Configuration](#) to learn more about the attributes.

Refer to [Set Up Identity Provider Front End](#) to learn about setting up the front end.

Set up Basic SAML SSO where AM acts as SAML SP backend to initiate the authentication request and receive user identity from IdP

1. Create a new Identity Provider (UAM as Service Provider) or edit DefaultIdentityProvider, using the info provided by IdP
2. Navigate to Configuration > Unified SSO > Federated SSO > SAML > Identity Provider
3. Provide the IdP public key info for signing verification, via (a) or (b)
 - a. Fill in the information for **Keystore**, **Keystore Passphrase**, **Key Alias**, which is the public key from IdP
 - b. Fill in the information of IdP public key in **IdP public key string**
4. Create a new Service Provider (UAM as Service Provider) for the SP web application
5. Navigate to Configuration > Unified SSO > Federated SSO > SAML > Service Provider
6. Fill in the information for **Id used as issuer**, an identifier of the SP. eg. sp.i-sprint.com
7. Fill in the information for **Assertion Consumer Service URLs**, which is the URL to which the SP web application wants the SAML response to be sent. eg. http://[host]/samlsp-sample-app/acs

AM is now ready to provide SAML SSO service as SP backend via APIs (ie authz/saml/authnReq/generate, authz/saml/authnResp/parse).

Refer to [SAML Web SSO Configuration](#) to learn more about the attributes.

User to always provide credentials (for session integrity and/or step-up authentication) for SAML SSO, when AM acts as IdP

1. **Set up Basic SAML SSO where AM acts as SAML IdP backend to authenticate and provide user identity info**, if not previously done
 2. Navigate to Configuration > Unified SSO > Federated SSO > SAML > Service Provider. Select the Service Provider (UAM as Identity Provider) for the SP web application
 3. For IdP-initiated SSO, set **Always Prompt Re-authentication for IdP-initiated SSO** to true
 4. (Optional) For SP-initiated SSO, set **Always Prompt Re-authentication for SP-initiated SSO** to true
-

i Note: Leave it as false for most typical use case, where it is at SP discretion if re-authentication is needed. Only set this to true if AM as IdP is to enforce tighter security over the relying party.

SSO through this IdP will require the user to re-authenticate to proceed.

Refer to [SAML Web SSO Configuration](#) to learn more about the attributes.

Refer to [SAML ForceAuthn](#) to learn about session integrity/step-up authentication.

Refer to [Service Provider-initiated vs Identity Provider-initiated](#) to learn about IdP-initiated SSO & SP-initiated SSO.

User to always provide credentials (for session integrity and/or step-up authentication) for SAML SSO, when AM acts as SP

1. **Set up Basic SAML SSO where AM acts as SAML SP backend to initiate the authentication request and receive user identity from IdP**, if not previously done
2. Navigate to Configuration > Unified SSO > Federated SSO > SAML > Service Provider. Select the Service Provider (UAM as Service Provider) for the SP web application
3. Set **Default 'forceAuthn' in SAML Request** to true

i Note: Authentication request initiated via this SP will contain parameter nameValue: forceAuthn=true. The forceAuthn value can be overridden via authz/saml/authnReq/generate API request parameter.

SSO for this SP will require the user to re-authenticate to proceed.

Refer to [SAML Web SSO Configuration](#) to learn more about the attributes.

Refer to [SAML ForceAuthn](#) to learn about session integrity/step-up authentication.
